Role-Oriented Service Architecture

© 2026 S++ project contributors. All rights reserved. Book copyright.

S++ (SPP): Modeling, Language Mechanisms and Engineering Practice

English edition · October 15, 2026

Monetary examples use US dollars (USD), with English participant names. Amounts are illustrative; no exchange-rate conversion is implied. The banking case retains its original currency-specific eligibility rules.

Contents

- Part I — The Basic Model of Roles and Services
- Chapter 1 — From Objects to Business Activities
- Chapter 2 — Service Equals Participants Multiplied by Behavior
- Chapter 3 — Roles and Business Identity
- Chapter 4 — Service Categories and Scenario Organization
- Chapter 5 — Quantum Behavior and Quantum Services
- Chapter 6 — Superposition, Commutativity and Entanglement
- Chapter 7 — Queries, Calculations, Validations and Ordinary Updates
- Chapter 8 — Reusing Services and Scenarios Through Role Polymorphism
- Part II — Service Cooperation and Execution
- Chapter 9 — Service Cooperation and Execution Order
- Chapter 10 — Parallel Execution and Service Call Costs
- Chapter 11 — Minimum Service Granularity and Deployment Principles
- Part III — Expressing Roles and Services in S++
- Chapter 12 — Responsibilities of SPP, APP and PHY
- Chapter 13 — Entity, Reference, Role and Contract
- Chapter 14 — Declaring Role Relationships with take … as
- Chapter 15 — Four Categories of Atomic Service
- Chapter 16 — Calculation Rules and Collection Participants
- Chapter 17 — Serial and Parallel Scenarios and Contract Formation
- Part IV — Consistency and Execution Guarantees
- Chapter 18 — Business Facts, Participant Identity and Storage Ownership

- Chapter 19 — Transaction Boundaries, Execution Plans and Deployment Capabilities
- Chapter 20 — Quantum Properties, Call Bounds and Conserved Settlement
- Chapter 21 — Rollback, Inverse Transactions, Reversals and Compensation
- Chapter 22 — Generation, Physical Resources and Runtime Configuration
- Part V — Separating Business and Technology
- Chapter 23 — Benefits of Separating Business and Technology
- Chapter 24 — Performance Through Caching and On-Demand Data
- Chapter 25 — Encryption, Fingerprints and Security Services
- Chapter 26 — Idempotency, Duplicate Guards and Reliable Execution
- Chapter 27 — Runtime Governance and Technical Evolution
- Part VI — Complete Petstore Practice
- Chapter 28 — Identifying Roles, Behavior and Services from Requirements
- Chapter 29 — Zero Opening, Funding and Inventory Receipt
- Chapter 30 — Quotation, Ordering, Payment and Shipping
- Chapter 31 — Cancellation, Returns, Reservations and Failure Handling
- Chapter 32 — Technical Configuration, Deployment and Integrated Verification
- Part VII — Advanced Cases and Architectural Methods
- Chapter 33 — Bank Deposits, Repayment Allocation and Reversal
- Chapter 34 — Migrating an Existing Object-Oriented System
- Chapter 35 — Architecture Review and Capability Boundaries
- Appendices
- Appendix A — The Service Model and Formal Derivations
- Appendix B — Conditions for Exchange, Reordering and Parallelism
- Appendix C — Performance Models and Experimental Methods
- Appendix D — Quantum Services and Conservation Mechanisms
- Appendix E — Language and Engineering Reference
- Appendix F — Sources and Verification Evidence

Part I — The Basic Model of Roles and Services

We begin with familiar object-oriented experience, move toward complete business activities, and then identify participants, roles and service responsibilities. “Service = participants × behavior” provides a common starting

point. Quantum services explain conserved changes; role polymorphism connects existing services to new participants.

Chapter 1 — From Objects to Business Activities

Both object-oriented and role-oriented modeling express entities and their behavior. Their emphasis differs in what they choose as the subject of modeling. Object orientation centers on entities and organizes properties and actions around them. Role orientation centers on business behavior and organizes participants around a complete business objective. The former asks what an object can do; the latter asks who participates in what capacity and how the activity is completed.

This book applies the modeling method to commercial service architecture. Part VII, Chapter 35 further examines its expressive scope and how it constructs complex data structures and algorithms.

Organizing Actions Around Objects

When a store accepts orders, concepts such as customers, products, inventory and orders appear. Object-oriented modeling usually establishes these objects first and attaches actions to them: customers maintain profiles, products adjust prices, inventory reserves quantities, and orders change state. Cooperation among objects completes the business process.

Actions belong to their objects. The model emphasizes their properties, capabilities and mutual calls. Interfaces, inheritance and composition help reuse capabilities, while objects and their capabilities remain the starting point for organizing business.

Organizing Participants Around Business Behavior

Role-oriented modeling first describes the complete behavior of purchasing: a buyer selects goods, reaches an agreement with a seller, and obtains goods or a delivery commitment. It then identifies roles such as buyer, seller and goods, and lets concrete entities play them.

A person purchasing household supplies and a company procuring office equipment are different entities. Both can play the buyer role if they satisfy the qualifications, facts and obligations required by purchasing. The behavior depends on the buyer role; the entity playing it can change.

One entity can also participate in different behaviors. Alex is a buyer when purchasing and a seller when operating a store. The participating identity changes with the activity; Alex has not become another person.

Role orientation therefore makes the business activity itself an explicit modeling subject. Its objective, relationships and completion conditions form the model's main thread. Objects join that thread as participants.

They still provide facts and carry actual changes, but the complete meaning of the behavior does not belong to one particular entity.

Starting with a Purchase

Alex buys two items. The store confirms purchasing eligibility, checks goods and quantities, calculates the agreed price, reserves inventory and forms an order. Customer and product data provide the facts; eligibility, price, reserved quantity and order terms belong to this activity.

Reading along the purchasing behavior, Alex participates as buyer, a product participates as saleable goods, inventory supports fulfillment, and the order records the agreement. Every query, calculation and update must be explained within that purchase: whose facts does it prepare, which agreement does it serve, and how does it help complete the whole activity?

Suppose two items have been reserved but the order could not be saved. Completing one operation does not mean the purchase has been established. Behavioral completeness requires us to explain whether the agreement was formed and what happens to existing changes. These requirements begin with the objective of completing a purchase and are then assigned to participant operations.

Actions and Behavior

An action is an operation an object can perform; behavior is an activity with internal logic and purpose. Changing a field is an action; confirming a payment is behavior. The former describes a local change. The latter also explains why it occurs, who participates, and what counts as completion.

This distinction prevents technical steps from becoming business services merely because they are operations. Serialization, sending a request and updating a row are runtime operations, but do not necessarily constitute complete business behavior. A payment cannot be considered complete merely because its request was sent; it requires an interpretable settlement result.

Different Participating Identities of One Object

A person may be buyer, merchant, payer or account owner in different activities. Their qualifications and facts differ. Buyers concern membership and purchasing relationships; merchants concern business operations; payers concern the funds used. Making one Person class carry all these rules concentrates unrelated business changes in one model.

S++ expresses participating identities through roles. Entities provide natural facts; roles explain how those facts enter particular services. Their relationship is developed in Part I, Chapter 3. For now, read the model by asking who participates in which behavior and in what capacity.

Constraining Models Through Results

Business modeling must answer what completion means. A successful order means that its agreed content and inventory reservation satisfy the terms. A successful payment means that fund changes and payment status satisfy the terms. These results let us judge decomposition, identify steps that must succeed together, and recognize technical failures requiring recovery.

Role-oriented service architecture retains the facts of the object world while expressing business activities, participating identities and cooperation explicitly. The next chapter connects them through a simple model: service equals participants multiplied by behavior.

Chapter 2 — Service Equals Participants Multiplied by Behavior

“Transfer” names a behavior. To understand the business activity it forms, we need to know who supplies funds, who receives them, the amount and the constraints. Conversely, placing two accounts together does not transfer money. Participants and behavior jointly form a meaningful business activity.

In “service = participants × behavior,” service has its broad, traditional meaning, as in order-placement or payment service: participants jointly complete business and produce an intended result. Multiplication emphasizes that both are necessary. Behavior needs participants to carry it; participants must enter behavior to produce a business result.

SPP further distinguishes **services** from **scenarios**. A service carries a defined business responsibility. A scenario organizes participants and services around a complete business objective. This chapter explains the common model and its mapping to those concepts.

A Simple Expression

Write a business activity as:

```
B = p × b
Business activity = participants × behavior
```

Here p denotes a participant or participant combination, b a behavior with business purpose, and B the activity with its objective and result. A complete service in the traditional sense can combine multiple activities. Appendix A gives the full notation and conditions.

Multiplication is a modeling expression, not numerical multiplication of account values and methods. Nor does it imply that all business steps obey ordinary algebra. It helps identify missing participants, behavior or results in a

definition.

Participants Determine the Concrete Meaning

Buyer-to-merchant payment, merchant-to-buyer refund and settlement-account-to-wallet funding all involve different relationships. Naming all of them pay obscures the differences. The model must identify who plays each role, which facts change, and which contract expresses the agreement.

Participants are also more than a collection of primitive interface parameters. An account number locates an account but does not fully express payer eligibility, account facts or confirmed identity. An amount must belong to an explicit payment agreement so the caller knows whether it is a quotation, payment or refund. A participant is therefore more than a conventional class: it needs the relevant role information.

From Complete Business to Services and Scenarios

A conventional order-payment service queries buyer, merchant and order, confirms eligibility, prepares the amount, transfers funds and records order state and a receipt. Together these activities fulfill the objective of paying an order.

Concept	Meaning in Order Payment
Service	A defined responsibility: query a participant, check eligibility, prepare an amount, transfer funds or save an order
Scenario	Organize buyer, merchant, order and payment contract; combine services; express sequence, transaction boundaries and completion requirements

The fund-transfer service describes a decrease in the buyer's wallet and an increase in merchant funds. The scenario also identifies the order, the source of its amount, eligibility and how fund changes and order records form the payment result. Fund transfer alone cannot replace the entire activity.

Analysis therefore starts with “the buyer and merchant complete order payment under a payment agreement.” We then identify service responsibilities and organize them in a scenario. The model begins with the objective; SPP expresses local responsibilities and overall cooperation separately.

Chapter 3 — Roles and Business Identity

A product in a catalog has a name, specifications and cost. When sold, it also has a selling price, listing status and sales text. Product and saleable goods are closely related but have different responsibilities. S++ uses Product for the product and Goods for its sales role.

Roles make participating identities part of the model. They can borrow facts from role players and own new facts arising from participation.

Identity and Eligibility

The same Person can play Buyer and Merchant. A person's name is a natural fact, membership tier belongs to the buyer identity, and store name belongs to the merchant identity. Creating a person does not automatically authorize operating a store; role eligibility must be established and checked separately.

Changes become easier to locate. Membership discounts concern the buyer role and pricing services, while customer-name maintenance remains in the customer domain. Roles connect activities to entity facts and help distinguish fact ownership.

Joint Role Playing

Some roles require multiple participants. A cart line involves both goods and buyer: the same goods have different quantities in different buyers' carts. Goods and buyer therefore jointly play CartLine, whose quantity is its own fact.

Joint role playing expresses relationship-based business. Ask which participants are indispensable to interpreting a role's facts. If cart quantity makes no sense without buyer and goods, its ownership should express that relationship.

Roles Can Play Other Roles

Goods entering an order form an agreed order line. Current selling price can differ from the agreed unit price because historical terms must be retained. Goods can play OrderLine, which owns agreed quantity, unit price and amount.

Alex orders “Kitten Food” at 60 US dollars. The store later renames it “Kitten Formula Food” and raises the price to 65 US dollars. The product page can show the new facts; Alex's old order must still record “Kitten Food” and 60 US dollars.

When goods play an order line, distinguish information read as current goods values from information recorded separately at settlement. The relationship expresses their connection, not an obligation to update historical order names and prices whenever goods change.

Roles and Object Types

An object type describes what an entity is; a role describes the capacity in which it participates. Alex remains one person while playing buyer when shopping and seller when running a store. Buyer and seller have different

requirements: confirming and paying for purchases versus providing agreed goods.

Conversely, different entity types can play the same role. A person purchases household supplies and a company procures equipment. Both can be buyers. Purchasing services depend on the role's facts, qualifications and obligations. The player is replaceable if those requirements remain satisfied.

There is no one-to-one relationship between entity types and roles. One entity can play many roles; one role can admit different entities. Role-oriented modeling identifies required participating identities from behavior before assigning entities to them.

Chapter 4 — Service Categories and Scenario Organization

Order payment combines business activities and technical support. Eligibility checks, amount calculations and fund transfer are business; request parsing, data conversion, communication protection and storage access make them executable. Separating these categories and then refining business responsibilities clarifies every service in a scenario.

Technical and Business Services

Business services directly express business objectives and rules: whether an order may be paid, how much is due, and how money moves from buyer to merchant.

Technical services supply execution means: HTTP reception, JSON parsing, message encryption and database connections. Changing the protocol should preserve the business meaning of payer, payee and amount; the technical implementation changes.

Classification depends on responsibility, not name. Checking a request signature is technical security; checking purchase eligibility is a business rule. Business services declare needed facts and results; technical services obtain facts, transmit information and protect execution under the agreed contract.

Four Categories of Business Service

Category	Question	Payment Example
Query	What facts do participants have?	Obtain order, buyer and merchant information
Validation	Are business conditions satisfied?	Check order ownership and payment eligibility
Calculation	What result follows from the rules?	Prepare the payment amount from the order

Category	Question	Payment Example
Update	Which business facts actually change?	Transfer funds and record payment state and receipt

Ordinary queries and updates are built-in system services whose implementations developers cannot alter. Business calculations and validations are called separately in scenarios. Queries complete facts, validations judge conditions, calculations prepare results, and updates apply changes. Calculating 100 US dollars due does not mean payment occurred; actual changes must complete.

The categories generalize. Registration queries existing records, validates eligibility, prepares content and establishes identity. Quotation queries goods and membership facts, checks conditions and calculates price, without necessarily making a settlement update.

Updating Descriptive Properties

Descriptive properties state what participants are and their information or status: a customer's address, a merchant's store name, or goods changing from listed to unlisted.

Numbers can also be descriptive. Changing a price from 60 to 65 US dollars changes quotation input; it does not move five US dollars out of an account. Classification depends on business meaning, not whether the field contains a string or number.

These updates still require protection against competing overwrites and may participate in complete activities. Their distinctive role is maintaining information or status. The consistency chapters discuss protection.

Updating Quantifiable Transaction Properties

Other properties represent transaction quantities, such as wallet balances and inventory. Their changes must identify amount, source, destination and permitted range.

Payment of 100 US dollars decreases the wallet and increases merchant funds by 100. Reserving two items decreases available and increases reserved quantity by two. Business concerns the transfer and both results, not merely assigning a new field value.

Suppose a wallet contains 500 US dollars and receives 20 after being queried. A 100-dollar payment should subtract from the actual 520, leaving 420. Writing the previously calculated 400 would erase the funding. Transaction updates must express actual deltas and use settlement mechanisms to handle competition, bounds and related changes.

A conserved change between a pair of quantum properties is the smallest quantum service. Later chapters explain source, destination and joint settlement.

Scenarios Organize the Services

Each category has its responsibility; scenarios combine them into complete business. Payment specifies facts to retrieve, qualifications to check, amount preparation, fund changes, order state and receipts.

One activity can use both update kinds. Payment moves funds and changes order state. The scenario explains how they jointly form its result; the technical layer supplies storage and execution guarantees.

This preserves both clear responsibilities and the whole objective. The next chapter introduces quantum behavior and services through changes in transaction properties.

Chapter 5 — Quantum Behavior and Quantum Services

Changing an address and transferring 100 US dollars both change data but impose very different demands. An address describes a customer; fund changes alter actual assets. Treating both as generic field updates hides the timing, reliability and concurrency requirements of transactions.

These differences explain why quantum services are defined and why their efficient, stable execution is central to asset-transfer systems.

Timeliness

An alternative contact address usually only needs to become effective before its next use. Even next-day propagation may meet requirements without affecting completed business. An address used by an active delivery has that delivery's timing requirements instead.

Payment often differs. Buyers wait for confirmation before collecting goods; merchants wait for funds before shipping. Without confirmation, neither can proceed. Transfers must quickly establish whether funds arrived, rather than merely report that an update request was submitted.

Transaction systems therefore reduce the time to a definite settlement result. Speed includes reliable confirmation: an earlier response that cannot say whether funds arrived does not accelerate completion.

Reliability

An unsuccessful address save often leaves the original address and permits another attempt. An unsuccessful transfer must establish whether money was debited, credited or transferred twice. Underpayment, duplicate debit or one-sided completion can cause actual losses.

If A transfers 100 to B, a debit without a credit is incomplete; two debits with one credit are also wrong. Checking that individual methods returned successfully cannot replace verifying their joint result.

If settlement completed but the response was lost, retrying must not transfer another 100. The system needs the original transaction identity and result confirmation. Reliability includes complete settlement, duplicate handling and post-failure confirmation.

Concurrency

Customers seldom change addresses, while one account may receive and make many payments concurrently. Popular goods may attract many buyers at once.

Competition affects the same resource. Two buyers cannot both reserve the last item. Two requests calculating from old balances and overwriting them may lose one change.

Transaction services must protect actual deltas and settlement bounds at high frequency without putting unrelated requests into one queue. Different accounts and products can be handled separately; competition on one resource needs correct arbitration.

Update Meaning and Conservation

An address update states the new information. A fund transaction also states source, destination and amount. A decrease of 100 at A and increase of 100 at B sum to zero. Moving inventory from available to reserved likewise identifies where quantity went.

The basic expression is the current delta and its related parties, not a previously calculated balance that might overwrite intervening transactions.

Conservation also requires real sources and destinations. Wallet funding must identify the funds' source; inventory receipt must identify supply. Otherwise the new number cannot explain how resources entered the system.

Failure Handling and Traceability

Without intervening changes, saving the same address again produces the same information. Repeating a credit produces another amount. Transactions need identities distinguishing retries from new business.

Reversal must use the original actual deltas while retaining later business. Restoring an entire old balance may erase subsequent receipts. Original identity, source, destination, amount and result must therefore be traceable.

Settlement and recovery are part of transaction responsibility, rather than an afterthought added upon failure.

Why Define Quantum Services?

The common problem is expressing a transaction as a clear business unit that cannot be arbitrarily split, with dedicated execution guarantees. SPP defines a quantum service as conservation between a pair of quantum properties: one decreases and the other increases within one business dimension. This is its minimum granularity.

For a transfer of 100 from A to B, the debit and credit can be analyzed separately, but a debit alone cannot count as a transfer. Smaller fragments cannot independently fulfill its objective. The quantum service preserves the complete meaning.

Starting at 1,000 and 200, successful transfer gives 900 and 300; rejection leaves 1,000 and 200. The partial state 900 and 200 is not complete settlement. Commit and recovery mechanisms provide the actual guarantee; Appendix D gives detailed arguments.

Efficient, Stable Settlement Is the Core Priority

In payment, accounting and inventory systems, queries, validations and calculations prepare facts and results; assets actually change at settlement. Faster pages and queries cannot compensate for settlement that queues indefinitely, duplicates execution or leaves results unknown.

Efficient execution processes valid transactions promptly under specified resources and expected concurrency. Stable execution avoids lost or duplicate changes, preserves bounds and conservation, and confirms or recovers results after faults.

Architecture should distinguish preparation from settlement, and arrange resources, scheduling and recovery around quantum services. Both efficiency and reliability matter; removing checks only produces apparent speed.

Expressing Them in Services and Scenarios

SPP uses quantum update for balance and inventory deltas; callers specify permitted bounds. Part IV, Chapter 20, develops properties, constraints and actual settlement.

Payment scenarios still organize order queries, eligibility, amount preparation, fund transfer and receipt recording. Quantum services focus on complete transaction changes; scenarios connect them to other responsibilities to complete payment.

Chapter 6 — Superposition, Commutativity and Entanglement

A major difference between quantum services and traditional updates is their treatment of values during transactions. A traditional update often obtains and locks the current value before calculating a definite new one. A quantum service expresses its delta; even after that delta settles, the observed property value cannot be determined solely from the earlier query and this delta.

S++ borrows superposition, commutativity and entanglement from quantum terminology to describe value states, execution order and related changes. We begin with balance updates and then discuss order and conservation.

Superposition Allows Value Uncertainty After Settlement

An account has 1,000 US dollars and this operation subtracts 100. One traditional approach locks the account with `select for update`, reads 1,000 and writes 900. Other requests cannot change that record during the protected update, so the transaction can determine the value it produces.

The quantum service expresses "subtract 100." Seeing 1,000 in a query does not lock that value until the transaction ends or require the eventual balance to be 900. Other requests may change it meanwhile.

Suppose another concurrent payment of 200 also takes effect:

```
Balance at query: 1,000 US dollars
This delta: -100 US dollars
Another concurrent delta: -200 US dollars
Balance after both settle: 1,000 - 100 - 200 = 700 US dollars
```

This operation's 100 has settled, but the later observed balance need not be 900. Other settlements and execution order affect it. Quantum services guarantee the delta, while allowing the resulting property value to reflect concurrent changes. Observation obtains the concrete facts.

That is the borrowed meaning of "superposition before observation." After this service settles, the property is not restricted to the value predicted from this operation alone. With no other change it can be 900; with the other debit it can be 700. This debit may have succeeded in both cases.

Superposition thus concerns value uncertainty after settlement and before observation. Confirmation answers whether the delta succeeded; observation answers the property's value at that point. One cannot replace the other. Execution must still enforce bounds, conservation and settlement. Appendix D12 elaborates.

Commutativity Means Order Does Not Affect the Joint Result

With 1,000 US dollars, service A subtracts 100 and B subtracts 200. If both settle, either order gives 700:

```
A then B: 1,000 - 100 - 200 = 700
B then A: 1,000 - 200 - 100 = 700
```

Each service expresses a fixed delta rather than making its intermediate balance a fixed input for the next service. Execution order can therefore be exchanged even when both affect the same quantum property.

Commutativity relates to superposition. Intermediate observations and which change occurs first may differ, but they do not determine already prepared deltas. After all settle, the joint result remains equal. This supports unordered and parallel execution. Appendix B0 proves the two- and multiple-delta cases.

Entanglement Becomes a Conservation Relationship

Wallet payment of 100 and merchant receipt of 100 sum to zero. Decreasing available inventory by two and increasing reserved by two also describes conserved transfer. The changes constrain each other; without the counterpart, the destination of money or goods is unexplained.

Money and inventory remain different dimensions even when both are quantum properties. They cannot be added to prove conservation. Products must also be checked under their own identities. A scenario can coordinate them, but conservation groups must reflect actual relationships.

Multiple Quantum Services Can Be Submitted Together

Commutativity enables several services to become one larger quantum request. Each has defined participants and fixed deltas; execution can collect them and submit them together.

A transfers 100 to B and 200 to C. Separate submission sends two requests and receives two results; merged submission carries both:

```
Service A: party A −100, party B +100
Service B: party A −200, party C +200
Merged request: {A, B}
Joint delta: party A −300, party B +100, party C +200
```

Since order is exchangeable, the sender need not await the first before sending the second just to preserve sequence. The executor can arrange them under a common boundary. The business model retains distinct responsibilities while communication is combined.

This reduces network round trips, packaging and protocol processing. Fixed communication overhead is especially worth combining when services are numerous and individual changes small. Commutativity provides the basis; Appendix D6 explains merged submission and delta aggregation.

Chapter 7 — Queries, Calculations, Validations and Ordinary Updates

Quantum services change quantifiable properties such as balances and inventory. Complete business also retrieves facts, calculates amounts, checks eligibility, and saves information and processing status. Scenarios organize these responsibilities; applications choose execution accordingly.

Payment queries buyer and merchant, validates the order, prepares an amount, transfers funds, and saves the order and receipt. Each category has a defined responsibility.

Most Queries Naturally Suit Concurrency

Ordinary queries and updates are built-in. Developers call them in scenarios but cannot alter their implementations. Separate calculations and validations express rules.

Queries obtain existing facts. Querying buyer data does not debit inventory; querying a price does not change the buyer. When inputs are ready and neither query needs the other's result, they can usually start together.

Petstore queries buyer and goods together for ordering, and buyer and merchant together for payment. The scenario uses completed participant facts afterward. Waiting overlaps, especially for database and external-domain access.

“Most” preserves dependencies. Goods cannot be queried from an order until that order identifies them. Queries also fill in-memory participants; concurrent writes to the same participant property need an explicit fill relationship. Inputs and fact-completion ranges determine concurrency.

A query also needs participant confirmation. Finding no buyer differs from failing to access the customer system. Unconfirmed information cannot be treated as complete calculation input.

Calculations Can Reuse Calculations

Calculations use prepared facts and formulas: line amount is unit price times quantity, subtotal sums line amounts, and membership determines discount.

These formulas can be separate and reusable. Petstore `quoteOrder` calls `priceLine` for order lines and sums their results. Results are written to participants and reused as shared facts.

A calculation may call another calculation, but may not query a database, call validation, update facts or start a scenario. Missing prices must be queried by the scenario first; validity of quantity must be checked there. Readers can then understand calculations from existing facts alone.

The calculation call graph must be acyclic. A calling itself, or A calling B and B calling A, is prohibited recursion. The compiler rejects a cycle even when a base case exists. This does not prohibit for or while iteration. A subtotal can be accumulated in a loop, including calls to a line calculation, provided the service graph has no cycle. Graph checking does not prove loop termination; loops still need clear stopping conditions.

Scenarios Organize Atomic Cooperation

An atomic service carries one defined responsibility. Ordinary atomic services cannot call other services or scenarios. Queries, validations, calculations and saves do their own work; scenarios organize business cooperation explicitly. Built-in queries complete participant facts; built-in updates save fully prepared facts.

Pure calculation-to-calculation reuse is the explicit exception. It remains calculation, not a hidden mixed query-validation-update process.

A payment scenario visibly arranges “query order, validate eligibility, prepare amount, transfer funds, save receipt.” The reader can see when funds change and which facts are saved. Call restrictions keep responsibilities and combinations visible.

Repository access within built-in queries and updates follows APP/PHY configuration. Business developers select participants and call positions; configuration supplies databases, mappings and protocols.

Validation Controls Continuation

Validation checks conditions on existing facts without modifying participants: payment eligibility, shipping status or a required return reason.

Ordinary validation success continues execution; failure ends its execution scope. Failed payment eligibility prevents amount preparation and debit. A validation surrounding a local scope can execute that scope if allowed or skip it and continue later work.

The scenario may explicitly request reversal of the current transaction on validation failure. Reversing existing changes differs from merely ending a scope. The scenario chooses whether rejection causes rollback; Part III presents the syntax.

Calculation answers “how much,” validation “may we proceed.” Calculations cannot hide business rejection inside formulas. Validations do not call each other; scenarios arrange them.

Calculations and Validations Suit Local Execution

With prepared facts, these services usually work entirely in memory. Line arithmetic and checking order status can execute in the scenario's process without remote calls for each formula or condition.

Both can be deployed locally. Ordinary business usually executes them directly in dependency order rather than creating a concurrent task for each small operation. Calculate lines, sum subtotal, then check payment conditions: clear flow avoids communication, scheduling and join overhead.

Different requests may still run concurrently. Sequential calculation within one request does not serialize the whole system. Large collections or expensive calculations can justify other execution strategies; Part II explains the criteria.

Inputs must be prepared first. Membership discounts need a membership tier: query obtains it and calculation uses it. This avoids hidden external accesses making local computation unexpectedly slow.

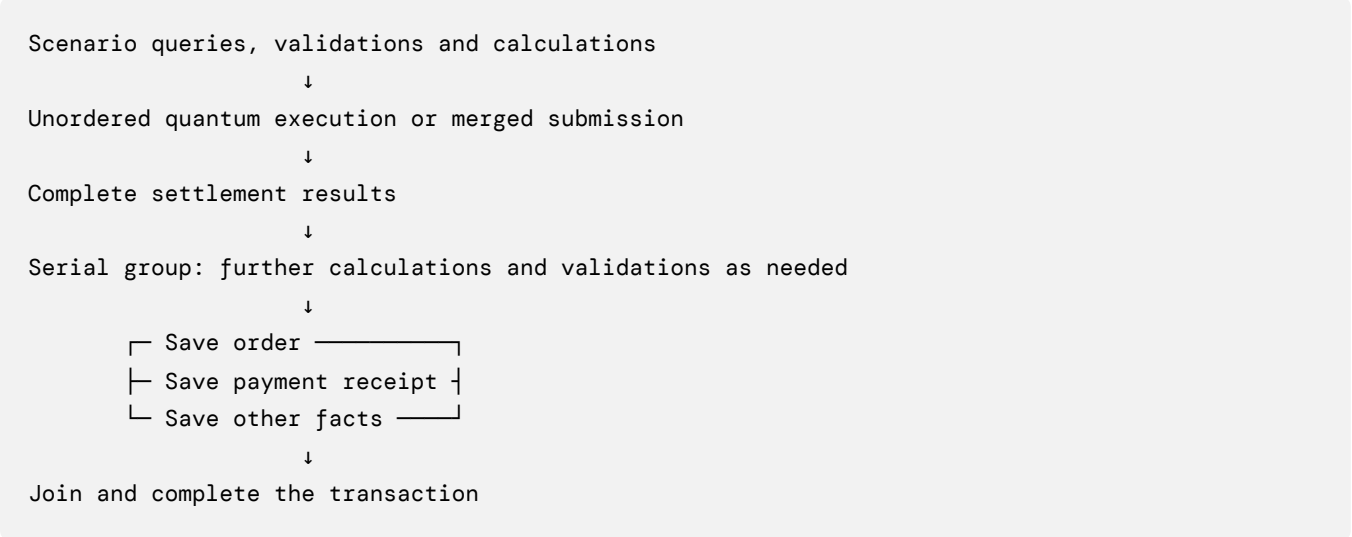
Ordinary Updates Run Concurrently After Quantum Services

Ordinary updates are built-in, with no developer-defined update implementation. They perform no business queries, calculations or validations and call no business services. The scenario supplies all information through queries, calculations and quantum services. Queries retrieve existing facts, calculations prepare facts to save, and quantum services apply actual deltas and complete settlement results. Updates save that prepared information.

For payment, the scenario prepares order, amount and receipt; the quantum service transfers funds. During ordinary saving, neither order save nor receipt save needs information from the other. Each uses complete facts independently.

Within one transaction boundary, the SPP compiler therefore schedules quantum transactions before ordinary updates. Ordinary updates do not change quantum inputs, deltas or results. Quantum transactions need not await saves and are themselves commutative, permitting unordered or merged submission.

If facts still require preparation after the quantum phase, the scenario first completes calculations and validations. Ready ordinary updates can then run concurrently. Exchanging their order changes neither prepared facts nor the result after all are saved. This is the basis of ordinary-update commutativity under the service rules and the generated quantum-first plan.



Developers plan calculations and checks preceding saves. After quantum payment, receipt balance, content and completeness may still need preparation. A serial group expresses “obtain quantum result, calculate receipt, validate receipt, save receipt.” A save cannot start before its inputs exist.

Serial groups establish phases; ready ordinary updates within a phase can still run in parallel. Built-in updates always save current complete facts. Appendix B12 explains exchange of updates, and B13 quantum-first scheduling and preparation phases.

Concurrent updates inside one transaction differ from independent transactions competing over the same information. Two profile requests still need version checks or conflict handling to prevent stale overwrite. Intra-transaction exchange does not remove inter-transaction protection.

The arrangement is clear: queries retrieve facts; calculations prepare results; validations control flow; quantum services apply deltas; ordinary updates concurrently save complete facts. Scenarios express dependencies and execution supplies scheduling and transaction guarantees.

This division makes concurrent execution a natural advantage. Explicit dependencies, built-in queries and updates, and quantum commutativity expose independent queries, merged quantum requests and concurrent saving. Fine responsibilities need not become a remote call apiece. Traditional microservices use fine distributed services for scalability and isolation but introduce communication and serial waiting. Part II explains how atomic-service concurrency reduces the response-time cost of finer granularity.

Chapter 8 — Reusing Services and Scenarios Through Role Polymorphism

Individual purchases and corporate procurement involve different entities but may both query goods, calculate quotations, validate eligibility and place orders. Services tied to concrete entity types invite duplicate logic when a new purchasing subject appears.

Role-oriented services depend on business roles. If different entities satisfy those roles, existing calculations, validations and scenarios can be reused. Change primarily concerns how entities enter roles and provide facts; existing workflows continue with the same roles and rules.

Role Polymorphism Makes Participants Replaceable

Object-oriented developers commonly use interfaces to accept different objects. Role polymorphism also admits different entities, but centers on the capacity in which they enter behavior. People and companies can be buyers; the same person can be a seller elsewhere.

A purchase scenario supporting both needs Buyer identity, delivery information and eligibility. Mapping personal and company names and retrieving delivery facts belong to role relationships and query completion. The scenario uses a prepared Buyer rather than separate individual and corporate scenarios.

“Playing the role well” means satisfying its facts and meaning: identity, property mapping, role-owned facts and rule inputs. Mapping the company name alone is insufficient. Other purchase facts must also be ready.

Reusing Calculations on Role Facts

Line pricing needs agreed unit price and quantity. Food, toys and household goods can reuse it when their entities provide the same meaning of goods facts.

Kitten food at 60 US dollars for two units yields 120; toys at 30 for three yield 90. Entities differ but the formula is the same. Unit price and quantity belong to the role, avoiding multiplication and rounding implementations for every goods type.

Petstore `priceLine` calculates an OrderLine's amount; `quoteOrder` reuses it for quotation. Reuse occurs both between calculations and when different entities enter the same role. A new product capable of playing Goods can reuse line pricing and quotation.

Likewise, discounts use Buyer membership tier and order amount, without testing the buyer's natural entity type.

Reusing Validation on Role Conditions

Validation expresses qualifications: positive purchase quantity, an unpaid order, or payer matching order buyer. These conditions concern roles, contracts and processing state.

A person buying two bags and a company buying twenty can use the same quantity check. It reads quantity without caring who supplied it. Payment eligibility can use the same Buyer-order relationship.

New entities can thus reuse existing validations. Scenarios call them in the same positions and branch on acceptance or rejection. Polymorphism reuses the rule; scenario organization preserves where it applies.

Reusing an Entire Scenario

Scenarios organizing reused calculations and validations can themselves be reused:

```
Confirm buyer and goods
          ↓
Validate purchase conditions
          ↓
Calculate line and order amounts
          ↓
Complete quantum transactions
          ↓
Further calculations and validations as needed
          ↓
Save complete business facts
```

Individuals and companies supply different entity facts; food and toys supply different product facts. After playing Buyer and Goods correctly, one scenario uses existing responsibilities to complete purchasing. Reuse covers whole behavior, not just a method or formula.

If quotation and ordering already support individuals, adding corporate procurement under the same prices, conditions and terms mainly requires the company to enter Buyer and provide facts. Quantity checks, line pricing, subtotal and saving remain reusable.

Roles connect reuse: entities satisfy roles, services use roles, and scenarios organize services. New entities can enter existing capabilities through existing roles.

Distinguishing Participant Changes from Rule Changes

First identify where change occurs. Replacing an individual with a company under unchanged rules preserves existing services and scenarios. Adding approval, credit purchasing or exclusive prices changes rules, so introduce the required roles, validations, calculations or steps explicitly. Existing applicable services remain reusable.

Business expansion starts by locating existing roles and services, explaining how new entities play them, and adding genuinely new terms. Capability can admit new participants without repeating logic for each entity type.

Role polymorphism supports reuse of fine-grained services and scenarios. The next part discusses the call costs of fine granularity and distributed deployment, and how concurrency and merged requests reduce scenario response time.

Part II — Service Cooperation and Execution

Explain each wait in ordering before deciding which queries and calculations can overlap. These three chapters discuss cooperation, call costs and deployment boundaries. Formal proofs and performance derivations appear in Appendices B and C.

Chapter 9 — Service Cooperation and Execution Order

Ordering needs buyer data and prices before calculating the agreed amount. It is tempting to write three successive steps: query buyer, query goods, calculate price. Only the calculation actually requires completed queries. The two queries can start together.

Explaining why each step waits is more useful than immediately selecting threads or network protocols.

Three Sources of Waiting

The first is business agreement. A store may require verification of returned goods before refunding. That result establishes refund eligibility; changing the sequence changes the policy.

The second is data dependency. Payment has no definite amount before the total exists; discount must await subtotal if it uses that subtotal.

The third is code arrangement. Buyer queries use membership credentials, goods queries use goods credentials, and neither needs the other. Sequential calls merely choose one execution strategy.

The third category, and other ready activities without mutual interference, can run in parallel. The first two require preserving dependencies or changing the business and calculation design.

Evaluating an Example

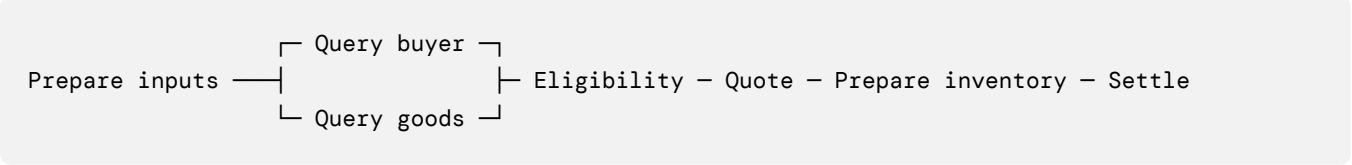
Suppose buyer query completes membership tier and goods query completes selling price. Quotation needs both. Concurrent queries do not omit either fact.

For calculations, A calculates line amounts and B totals them. B cannot start by reading unfinished lines. Different lines may be calculated separately before aggregation, but multiple threads must not arbitrarily overwrite one subtotal.

Queries also write memory. If both fill the same Buyer's name, the last write determines the result. Examine actual field-completion ranges.

Identifying Join Points

Arrows show waits in a simplified order:



Joining means both queries have completed with usable results. Missing identity or technical failure prevents normal quotation. Parallel plans must also identify branches that can stop and effects already produced on failure.

Updates Need Further Checks

Two requests each read a wallet of 100 and want to debit 80. Preparation can overlap, but settlement must enforce the lower bound so at most one succeeds. Earlier reads and scheduling cannot replace settlement

protection.

Wallet debit and merchant credit must also form one payment result. Starting both concurrently still needs a mechanism guaranteeing their joint outcome.

Ordinary updates have a different arrangement. Under Part I, Chapter 7, the scenario supplies all information through queries, calculations and quantum services. After quantum execution within one boundary, ordinary updates can concurrently save complete facts and join to finish the transaction. Order and receipt saves need not wait to obtain business information from each other.

The method is: identify inputs and outputs, explain waits, then check shared writes and settlement. Appendix B gives complete conditions, proofs and counterexamples.

Chapter 10 — Parallel Execution and Service Call Costs

Traditional microservices often seek scalability, isolation, independent deployment and scaling by dividing responsibilities into smaller services across processes or nodes. Responsibilities become more focused, but a complete scenario traverses more remote calls.

Smaller granularity may shorten individual business work while increasing the relative cost of communication, encoding, queueing and coordination. Serial calls accumulate waits and can substantially increase response time. More required remote dependencies also increase opportunities for a failure preventing the whole activity, pressuring scenario availability.

SPP overlaps atomic services to reduce response-time growth caused by finer granularity, retaining detailed business expression while overlapping independent waits. This chapter addresses response time; concurrency does not fix the availability issue introduced by those dependencies.

How Fine Granularity Extends Response Time

Assume ordering queries buyer in 40 milliseconds, goods in 70, then calculates in 10. Sequential execution takes 120. These illustrative figures explain structure, not measured performance.

Only quotation needs both query results. Neither query needs the other, so sequencing them adds waits that could overlap.

Request encoding, round trips and service queues extend serial waiting further. Computation may be brief while most scenario time is spent calling and waiting.

How Atomic Concurrency Reduces Waiting

Scenarios explicitly express participants and dependencies. Ready independent queries run together, followed by quotation:

```
Serial: buyer 40 — goods 70 — calculation 10 = 120 ms

Concurrent: buyer 40 ┐
                     ├─ calculation 10 ≈ 80 ms + scheduling/join overhead
            goods 70 ┘
```

Work remains the same, but waiting overlaps. The scenario waits for the slower 70-ms query, then calculates for 10. It mitigates accumulated serial-call latency.

The basis is clear service responsibility: built-in queries complete facts; calculations and validations execute separately; quantum transactions commute; prepared ordinary updates save concurrently. Explicit dependencies preserve required waits and overlap independent work.

Reducing Serial Waiting and Repeated Communication

Concurrent queries overlap remote waits. Quantum commutativity permits unordered execution and merged requests. Complete-input ordinary updates save concurrently. Calculations and validations normally execute locally in dependency order, avoiding remote calls for tiny formulas and conditions.

Business responsibilities can therefore be fine-grained without sequential waiting for every service. Concurrency overlaps waiting, merging reduces repeated communication, and local execution reduces small-service overhead. Together they shorten the path.

Preserve Dependencies and Measure Gains

Real dependencies remain. Subtotal needs line amounts. If receipt preparation follows quantum settlement, a serial group must arrange calculation and validation before saving.

Concurrency also needs resources. A database processing one request at a time still queues both queries internally. Insufficient connections or threads can worsen contention. Compare identical rules and commit guarantees, and measure response time, throughput and resource use.

Role-oriented architecture naturally organizes concurrency by responsibility and dependency, reducing latency growth while preserving fine modeling. Appendix C gives time models, resource conditions and experiments; the next chapter relates service to deployment granularity.

Chapter 11 — Minimum Service Granularity and Deployment Principles

Before asking how finely to split services, define the lower boundary. In SPP, a quantum service is conservation between a pair of quantum properties. Other services have a minimum granularity of an operation on a role. This business model does not split them further.

The boundary determines responsibilities; deployment determines locations and resource use. Deployment still depends on dependencies, communication cost and scaling requirements.

A Pair of Conserved Properties Is the Quantum Minimum

A transfers 100 to B: A decreases by 100, B increases by 100.

```
A.balance: -100 US dollars
B.balance: +100 US dollars
Sum of changes: 0
```

Keeping only debit or credit destroys the transfer's complete meaning. The pair's conserved change is the minimum; a single property's delta is not a smaller independent quantum service.

Inventory reservation similarly decreases available by two and increases reserved by two for the same goods. The properties may belong to one role or separate participants. Actual source and destination within one dimension are decisive.

Combining quantum services into larger requests does not change that minimum definition. Each basic change remains a pairwise conserved service.

Other Services Use Role Operations as the Minimum

An ordinary query completes a role's facts, an update saves them, and calculation or validation performs a defined operation concerning a role: query Buyer, save Buyer, calculate OrderLine amount.

A role may have many properties, while an operation uses only some. Eligibility may read only buyer status but still judges Buyer participation. Line pricing uses price and quantity but still calculates OrderLine amount.

Complex calculations reuse calculations and scenarios organize multiple role operations. Splitting a field read, multiplication or comparison into a business service goes beyond SPP's minimum; these are expressions inside the operation.

Comparing Property-Level Object Operations

Object-oriented code can expose `getName`, `setName` and another pair for address. Individual property reads or assignments can be methods, making code-level operations as small as one property.

SPP defines business services as role operations. Querying and saving Buyer complete and persist required facts. Names and addresses remain properties, but their accessors are not smaller business services. Queries and updates are built-in; scenarios call them and calculations or validations express rules.

SPP therefore has a definite endpoint: pairwise conserved quantum changes or role operations, rather than continuously shrinking the amount of data inside a method.

Query Properties According to Use

Role granularity does not imply retrieving every property throughout the entire role-playing chain. Business granularity and loaded data volume are separate.

At compilation, property use in scenarios, calculations and validations can be collected into a required set. Generated queries need only complete those facts from their mapped sources.

Buyer may be played by a customer entity. Quotation needs tier, purchasing eligibility needs status, ordering needs address. Queries can obtain those and necessary identity without unused customer data. Buyer remains the operation unit while database reading, remote transfer and mapping volume decrease.

Reduce Repeated Reads with Multiple Cache Levels

Request caches and shared process caches can reuse facts. One scenario may use an entity repeatedly; different requests may share relatively stable information under an appropriate policy.

Local caching is particularly useful for remote entities. Names and categories need not be fetched from the product domain every time. Cache required facts and complete local roles through the mappings.

These familiar optimizations can be implemented by built-in queries and APP/PHY configuration. Scenarios continue to call role queries. Refresh and invalidation follow requirements, while real money and inventory settlement still uses authoritative quantum mechanisms.

On-demand properties reduce data per read; caches reduce repeated reads. Both mitigate wide roles and long chains without splitting into property services. Appendix C13 states detailed conditions.

Prefer Local Calculations and Validations

Line pricing, discounts and eligibility can retain separate definitions yet run inside the scenario process. Prepared facts avoid network round trips for brief formulas and conditions.

Definitions remain independently reusable and composable, while one deployment unit can host many services. The model retains minimum responsibilities; runtime groups them by cost.

Arrange Queries, Quantum Transactions and Saves by Dependency

Queries run locally or remotely according to fact location. Independent role facts can be queried concurrently. Separate scaling or team ownership may justify remote product or customer access.

Within a boundary, quantum transactions precede ordinary updates and can run unordered or merged. After results arrive, serial groups arrange further calculation and validation, then ready updates run concurrently. Deployment must support these dependencies while controlling communication and coordination.

Service and deployment granularity need not coincide. Customer, product and shop responsibilities can share a process or be separated. “Co-location” simply means running multiple services in the same process, such as one Java application calling them locally. Responsibility ownership remains; paths and resources change.

Deployment Must Honor Transaction Boundaries

Petstore develop hosts four domains in one process with LOCAL. testing separates customer, product and shop applications. The business model is the same, but communication and commit conditions differ.

Shared-process services using a common resource can handle quantum and ordinary changes in one local transaction. Separate processes require supported coordination or explicit phased settlement and recovery. Merely naming deployment distributed provides no joint commit guarantee.

Model at the defined minimum first; deploy by business dependencies, cost, capacity and transaction capabilities. This preserves detailed role services while reducing scenario response time.

Part III — Expressing Roles and Services in S++

SPP, APP and PHY separate business from technology and logic from physical implementation, clarifying the work of requirements specialists, technical developers, operators and AI. We then express the model in source. Goods, carts and funding examples introduce role playing, atomic services and scenarios as checkable declarations.

Chapter 12 — Responsibilities of SPP, APP and PHY

S++ layers separate business from technology and logic from physical implementation. The first lets people with different skills handle business development and technical adaptation. The second separates technical development from deployment and operations.

For payment, business specialists care about parties, amount, eligibility and joint results. Technical developers handle storage and external-system integration; operators choose databases, locations and resource configuration. Combining everything in one program makes all participants understand and change mixed logic.

SPP, APP and PHY respectively carry business expression, application technical contracts and actual physical configuration, connecting the work through explicit agreements.

Let Business Specialists Express Business

SPP defines participants, roles, contracts, calculations, validations and scenarios. Payment amounts, discounts, shipping eligibility and conservation belong here, as do required transaction boundaries.

Business development need not also write SQL, message conversion, networking or transaction-framework code. Built-in queries and updates, business calculations and explicit scenarios let people inspect requirements without understanding large implementations.

For “members receive ten percent off,” a requirements specialist can inspect membership role, formula and quotation flow. Buyer-customer role playing is expressed in SPP; integrating the customer system and choosing retrieval protocols is technical work.

Business development can even be assigned directly to people who understand requirements. After learning roles, services and scenarios, they can produce compilable models. Technical developers focus on connecting those expressions to actual systems.

Roles and contracts become shared interfaces: business specifies payer, payee and amount meaning; technical development supplies their facts and execution.

Separate Technical Development from Operations

Technical logic states required capabilities; physical configuration supplies them in an environment. APP organizes repository mappings, bindings, interfaces, duplicate protection, numbering, logical resources and environment deployment. PHY selects databases, protocols, numeric representations and runtime resources.

Accessing customers, saving orders and using transaction resources are application contracts. Actual development, testing and production addresses, connections and parameters are physical choices.

Developers build and adapt capabilities; operators configure and manage resources. Switching customer endpoint or migrating a database should not rewrite payment or discount formulas.

Some capabilities, such as a new numeric representation or transaction protocol, first require an adapter. Once available, operators select and configure it. Development and resource management thus have distinct locations.

How Three Layers Complete Payment

Layer	Content	Primary Responsibility
SPP	Buyer, merchant, agreement, qualifications, amount and transaction boundary	Business developers or requirements specialists familiar with the language
APP	Storage mappings, bindings, interfaces and technical contracts	Technical developers and integrators
PHY	Databases, endpoints, resources and physical representations	Operators, with adapters supplied by technical developers

SPP expresses buyer -100 and merchant +100 and the order-receipt result. APP organizes storage and integration, selecting execution that honors the boundary. PHY supplies actual resources.

Order numbers with direct business use can belong to SPP. Internal storage keys are supplied by compiler and system, without an extra business ID. Rounding is business; machine representation and storage capacity are physical. These distinctions preserve meaning while separating maintenance.

AI Maintains Less Source; Tools Produce Deterministic Output

Traditional AI-generated payment code may mix business decisions, objects, conversion, repositories, remote calls, transactions and deployment. More code creates more consistency relationships to preserve and more locations to find when requirements change.

S++ gives AI clearer tools. Business changes concentrate in SPP, integration in APP, environments in PHY. AI modifies relevant source rather than recreating all implementation layers each time.

Compilers, generators and runtimes produce repeated model, mapping, query/save integration, call, interface and storage code from declarations. The same source, configuration and tool version use the same rules to produce corresponding output.

The amount of freely generated and maintained AI code can therefore decline substantially. Relationships formerly written repeatedly and manually checked are generated and validated by tools. Fewer simultaneous choices reduce opportunities for omissions, duplication and cross-layer inconsistency.

A discount change alters calculation while tools maintain models and infrastructure. A protocol change alters adaptation while payment retains its roles and scenarios. Deterministic transformations let AI focus on requirements, modeling and necessary adaptation.

Tools still execute the supplied expressions. Business correctness and adapter guarantees require review and validation by their owners. Layers locate these concerns and reduce mixed business-technology-deployment-operations logic maintained by AI.

Human Focus and Quality Control

SPP contains business models, rules and flows. Human review can directly check parties, roles, calculations, conditions, settlement and failure results. Integration and deployment have their own places rather than being traced during every business review.

For AI-modified member discounts, specialists follow Buyer tier, formula and quotation to verify applicability, percentage and shipping. Readable expressions map to requirements.

Humans supply rules, exceptions and expected results; AI expresses them as roles, services and scenarios; tools generate implementation. This collaboration lets business omissions be found directly instead of reconstructed from mixed code by technical staff.

Each business change focuses review on SPP and results; technical and operational changes are reviewed by corresponding specialists. Tool rules and validation protect repeated generated code, reducing exhaustive review of every generated line. Toolchain or adapter changes receive targeted review of their own behavior.

Reduced code also narrows review: people can see what business AI changed, why, and whether results meet requirements. Expression, deterministic generation and real validation together support quality control.

Locate a Change in Its Layer

Membership discounts change SPP calculations; new customer systems change APP adaptation; production endpoints and connections change PHY.

Requirements specialists, developers, operators and AI have clear scopes, connected by shared contracts. Business stays readable, integration explicit and resources checkable. The following chapters introduce source expressions. Part V, Chapters 23–27, extends the benefits and technical services for performance, security, reliability and governance.

Chapter 13 — Entity, Reference, Role and Contract

Types answer different questions. Entity is a natural entity owned by the current domain; Reference points to an external-domain concept; Role is participating identity; Contract records an agreement formed by a scenario. Together they explain meaning and lifecycle ownership.

Natural Entities and External References

The customer domain owns Person and the product domain Product. The shop uses their facts, but the original domains own their lifecycles. Reference makes origin explicit; APP and PHY determine integration.

References can target provider entities, roles or contracts. They identify fact sources without taking lifecycle maintenance away from providers.

Roles and Scenario Contracts

Buyer expresses shopping identity and Goods sales identity. ShopOrder records ordering terms; PaymentAgreement payment terms. Later payment, shipping and return scenarios can reference an order without repeating ordering.

A short Petstore contract is:

```
Contract WalletFundingAgreement {
    Buyer buyer;
    SettlementAccount source;
    BigDecimal(19, 2) amount;
}
```

Participants and amount describe one funding activity. The contract records terms; services implement changes.

Constituent Facts and Independent Participants

Order lines retain goods, quantity and agreed unit price. Total is a constituent fact of the agreement, not necessarily a separate amount role. Independent roles require real participation and lifecycle meaning.

Aggregating lines in a contract does not mean the contract plays each line. Composition and role playing are separate relationships.

Internal Keys and Business IDs

Conventional developers often add a unique `id` to identify objects and database records. S++ does not require a business field merely for technical location: the compiler provides a business-invisible internal entity key. Technical uniqueness does not require a business ID.

Habitual fields such as `id` or `customerId` can imply nonexistent business credentials. The compiler checks suspicious habitual IDs and warns developers to confirm real meaning.

Where an ID truly serves business, `business` explains its purpose and removes the warning. A receipt identifier used by customers to query results is one example:

```
Entity Receipt {
    business("Business receipt number used by customers to look up and verify the outcome")
String receiptId;
}
```

The customer uses this value. If its only purpose is database uniqueness, the internal key already serves that responsibility. `business` explains why a seemingly technical ID belongs in the business model.

Its purpose is resolving ambiguity. Review still asks who uses the number, in which activity and how it supports processing or verification. “Internally unique” describes technology, not a business credential.

Models are organized by membership, goods, cart, orders and other functions. See the Petstore source guide.

Chapter 14 — Declaring Role Relationships with take ... as

Goods defines facts required by a sales role, but not who plays it. `take ... as` declares players, mappings, borrowed facts and role-owned facts. Players follow `take`; the role follows `as`. `take Product as Goods` means Product plays Goods.

One-Party Role Playing and Mapping

Petstore maps products to goods:

```
take Product as Goods {
    mapsame;
    String sku = productCode;
}
```

`mapsame` explicitly enables compatible same-name mappings. If Product and Goods both have name, goods can obtain it from the product. Differently named fields need explicit mapping, here Product.productCode to Goods.sku.

This is not the default. The following declares only sku's source, without automatically mapping name:

```
take Product as Goods {
    String sku = productCode;
}
```

A `take ... as` shorthand without a block also only declares the relationship. `takeall` explicitly imports player properties and creates mappings; it differs from the shorthand.

Mapping also determines ownership. Borrowed facts remain owned by their player; added or explicitly redefined facts belong to the role. Queries complete both, but saves must respect ownership boundaries.

Querying the Chain and Filling Properties

Goods queries retrieve role facts and the corresponding Product. Declared mappings then fill Goods from source facts.

“Product.name was queried” and “Product.name supplies Goods.name” are different steps. Retrieval obtains data; mapping chooses its use. With mapsame the name can be completed. With only sku mapping, querying Product creates no name mapping. Recursive chain querying therefore does not make mapsame optional by default.

Joint Role Playing

Goods and buyer jointly play a cart line:

```
take Goods, Buyer as CartLine {
    Goods goods = Goods;
    Buyer buyer = Buyer;
    int quantity;
}
```

The declaration identifies two players. `Goods goods = Goods;` maps the entire goods participant into the cart line: the left Goods is the property type, goods its name in CartLine, and the right Goods the declared player.

`Buyer buyer = Buyer;` is analogous. The right side does not create a new buyer or read a same-name property. The generated mapper supplies the player object; compatible undeclared target properties are added by the compiler.

```
CartLine
├─ goods: Goods, the item selected
├─ buyer: Buyer, who selected it
└─ quantity, selected for this line
```

Quantity is CartLine's own fact and must be declared in the role definition. Access names as `cartLine.goods.name` and `cartLine.buyer.name`, even if both participants have name. This is business structure; APP determines storage.

Alex and Jordan select the same goods, two and three units respectively. The lines share goods but differ in buyer and quantity. CartLine preserves which buyer selected which goods, while APP locates each line by both identities.

Whole-Participant Mapping Versus takeall

The current keyword for bulk importing player properties is `takeall`; there is no `mapall`. It creates a different structure from whole-participant mapping.

For Goods with sku and name:

```
take Goods as DisplayGoods {
    takeall;
}
```

`takeall` imports them into DisplayGoods, adding missing properties and mappings. Access becomes `displayGoods.sku` and `displayGoods.name`, without a goods grouping property.

Declaration	Role Structure	Name Access
`Goods goods = Goods;`	Goods-typed property preserving the entire participant	`cartLine.goods.name`
`takeall;`	Player properties imported into the role itself	`displayGoods.name`

A cart line needs both goods and buyer, and whole mapping gives each a distinct place. Applying takeall to both does not automatically create goods and buyer groups. Same-name properties need explicit source distinction.

`mapsame` maps existing compatible same-name role properties; `takeall` also imports properties the role has not declared. Choose whether business needs whole participants or selected properties before declaring mappings.

Role-Owned Facts

A merchant played by Alex may borrow personal facts and own store name “Alex's Pet Store” and operating funds. Personal-data changes are obtained through mappings; changing store name changes merchant facts, not Alex's personal name.

Distinguish borrowed and identity-specific facts. Contracts retaining historical settlement information are a separate requirement, discussed in Part IV, Chapter 18.

Legality and Persistence

The compiler checks that players are defined legal participant types, targets are roles, and property types satisfy the relationship. The current strict check allows Entity, Reference and Role but rejects Contract as player, when `strictRoleOnlyBusinessModel` is enabled.

Contracts can still be passed as service or scenario context for calculation, validation and later processing. Refunds can read the original order while buyer and merchant participate. Containing lines does not mean playing lines; composition differs from role playing.

A string name cannot become a number in a same-name mapping. Whole-participant mapping must reference a declared player and match the target type. These errors belong at compilation, not at query or save time.

Legal declarations enable generated types, mappings and metadata, but do not create participants. Person may play Merchant without automatically establishing a merchant record for Alex; opening and saving remain scenario activities.

Persistence respects ownership. Ordinary role updates maintain role facts; customer or product data uses its appropriate built-in services. Compile-time legality and runtime participant-specific querying and saving jointly support the relationship.

Chapter 15 — Four Categories of Atomic Service

Ordinary business atomic services are query, check, calculate and update. They retrieve facts, judge conditions, form candidate results and apply changes. Scenarios combine them into complete activities.

Queries Complete Facts

Ordinary query is built-in. Scenarios call `findBuyer` and similar services directly; APP/PHY retrieve facts and complete input participants without redeclaring standard queries. There is no business return value such as `buyer = findBuyer()`. Identity confirmation is execution state, not inferred from nonempty fields. Completion need not load the whole role chain; property analysis and caching can optimize it as described in Part II, Chapter 11.

An omitted instance name defaults to its type. Multiple participants of one type need aliases. Omitted arguments require unique matching by complete type and collection dimension; ambiguity requires explicit selection.

Validation Checks Eligibility

check reads facts and uses require for requirements without changing participants. Ordinary failure ends its lexical scope, not automatically a transaction rollback. Supported explicit control requests reversal.

Visible checks clarify admission. Actual quantum bounds must be enforced at settlement, not substituted by earlier balance reads.

Calculations Prepare Results

calculate changes candidate facts through formulas. It may call pure calculations, not query, check, update or scenarios to hide side effects.

```
atomic calculate priceLine(OrderLine) {
    OrderLine.amount = roundHalfUp(
        OrderLine.unitPrice * OrderLine.quantity, 2);
}
```

This definition is from pricing source, with line wrapping for presentation.

Updates Apply Changes

Ordinary update is also built-in. After preparing complete facts, scenarios call standard insert, update or delete services suffixed by participant type. Current participants are passed; the framework retains technical baselines without a business before snapshot. Business calculations and checks stay outside the built-in update. Quantum updates describe special-property deltas and settlement requirements.

The categories clarify dependencies. Preparation and checks can often precede the settlement boundary, leaving protected updates inside. Business and concurrency constraints still matter; calling each category once is not automatically complete business.

Chapter 16 — Calculation Rules and Collection Participants

Calculations express amounts, quantities, eligibility and policies. S++ retains familiar conditions and loops while separating mathematical meaning from physical representation. Formulas show business relationships; PHY supplies representation.

Make Rounding a Business Rule

Line amount uses unit price times quantity, rounded to two decimals as agreed. `roundHalfUp` states the rule. Without a business rounding requirement, storage capacity or default precision must not silently change values.

BigDecimal is one physical option. Some operations still require explicit rounding or another representation; exact decimal does not finitely represent all mathematical results.

Rules Execute in Defined Order

Petstore calculates lines and subtotal, then membership discount, shipping and total. These depend on facts at the current execution point.

if, switch, while and for provide deterministic flow, not a repeatedly firing rule agenda or arbitrary Java calls. Formulas remain within supported expressions and call contracts.

Collections and Internal Lines

Participants can be one-dimensional collections such as Goods[]. Queries complete them, calculations traverse goods and cart lines and form new internal order lines. Scalar input instances cannot be replaced; collection inputs retain their container while contents change.

Syntax accepts `Goods[][]`, but current query, update and calculation implementations do not support multidimensional participant collections. Examples use one dimension; implementation will expand if practical demand emerges.

A schematic subtotal service, omitting domain and other definitions:

```
atomic calculate sumOrderLines(ShopOrder) {
    ShopOrder.subtotal = 0;
    for (OrderLine line : ShopOrder.lines) {
        ShopOrder.subtotal += line.amount;
    }
}
```

The loop is sequential. Parallel line processing requires explicit branch results, aggregation and execution support.

Candidate and Settled Facts

A calculation may mark an in-memory candidate paid, but only protected updates make it an actual result. Technical failure cannot publish a candidate as settled.

Separation allows independent formula verification and shorter transactions. Version or settlement guarantees must still prevent competition after preparation from causing stale overwrite.

Chapter 17 — Serial and Parallel Scenarios and Contract Formation

Atomic services explain local responsibilities; Scenario expresses one defined business activity. Readers should see participants, qualifications, dependencies and commit boundaries without tracing hidden calls through layers.

Reading a Funding Scenario

From wallet funding source:

```
Scenario serial fundShopWallet(
    Buyer, SettlementAccount, WalletFundingAgreement) {
    checkBuyerCredential; checkFundingAmount; prepareWalletFunding;
    parallel { findBuyer; findSettlementAccount; }
    identify(Buyer); identify(SettlementAccount);
    transaction {
        transferWalletFunds bounds {
            SettlementAccount.balance: [null, null];
            Buyer.wallet: [0, null];
        };
    }
}
```

Inputs identify buyer, source account and agreement. Credentials and amount are checked, relationships prepared, then both sides queried concurrently. After identity confirmation, the explicit transaction executes fund transfer with bounds declared at the call.

Contracts in Scenarios

WalletFundingAgreement expresses parties and amount; ShopOrder ordering; ReturnRequestAgreement a return request. A contract corresponds to a defined activity, not an arbitrary wrapper for a list or return object.

Later activities reference existing contracts. Payment uses an order; returns use the original order and reason. This preserves relationships without invoking the original scenario again.

Explicit Orchestration and Automatic Binding

Argument-free calls reduce repeated names without removing participants. The compiler finds unique type bindings; multiple instances need explicit arguments. Readability cannot depend on guessing ambiguity.

Serial groups express order and parallel groups overlap execution. Inputs and effects must be established first; plans also check deployment support and recovery capabilities.

A Reviewable Business Entry

A scenario should correspond to a named activity. Packing many actions into a generic command string hides eligibility and participant scope inside branches. Separate payment, shipping and return entries state their facts more clearly.

Scenarios turn combinations into readable business narratives. The next part explains identity, storage, transactions and resources that honor them.

Part IV — Consistency and Execution Guarantees

Correct payment formulas are insufficient: participants must be located correctly, competing updates protected, and failures explained. Fact ownership, transactions, quantum settlement, recovery and generation show how those requirements receive real guarantees.

Chapter 18 — Business Facts, Participant Identity and Storage Ownership

Buyer may contain customer name, membership tier and wallet balance, but these facts need not share a storage owner. Queries complete them; saves respect their respective ownership.

Borrowed and Owned Facts

Customer name is borrowed through role playing and maintained by the customer domain. Tier belongs to Buyer; wallet settlement belongs to quantum storage. Ordinary role insert, update and delete write only their own table, without cascading to players.

This prevents goods updates altering products and cart saves implicitly modifying buyers. Entity maintenance is explicit through the original domain's services.

Contracts Retain Settlement Facts

An order for “Kitten Food 1kg” must retain that name after the merchant renames it “Kitten Formula Food 1kg.” Current product pages show the new name; the old order describes what was purchased.

Current names and agreed contract names have different duties. Order queries must not overwrite historical names from today's products. APP storage and mapping configuration implements this requirement; ordinary role

relationships alone do not imply it.

For example, a bank's failure-disposition record must preserve the deposit or reversal contract at the time of failure, including the system-provided cause, service, phase and recovery status. Saving only an association and later querying current facts can change the record's meaning as balances or contracts change.

APP `snapshot` now preserves explicitly declared associated participants and collections as historical snapshots. Queries do not continue filling those snapshot subtrees through role or association chains, and current quantum balances do not overwrite historical balances inside them. Ordinary live associations still follow their normal completion rules. APP configuration distinguishes these uses; see the bank snapshot verification.

Identity Is Not the Whole Fact Set

Wallet and name may change while buyer identity remains. APP locates participants through keys, business credentials or combinations. Comparing all facts serves other questions, such as changes in candidate content.

The compiler supplies business-invisible entity keys, maintained by internal metadata, avoiding habitual business id fields. Real order and membership numbers enter the model through their actual use. Versions, duplicate keys and repository location each have technical responsibilities.

Query Completion and Baselines

Queries and updates are built-in. APP/PHY determine completion, baseline recording and version comparison; scenarios arrange calls and rules.

Queries can retain an internal baseline for later saving. Version protection compares the version originally queried, rather than reading a fresh version immediately before overwriting a competitor.

If another request modifies facts during candidate calculation, saving reports conflict. This shortens lock time but requires explicit rejection when baseline is missing or identity changed.

Master-Detail

An order is Master; each line is Detail, recording quantity and agreed unit price. Cart and cart lines use the same structure.

The example preserves detail identities and completes members and related participants via query metadata. Saving Master does not replace explicit insertion, modification or deletion of Details.

Ordering explicitly deletes old cart lines, inserts agreed order lines and saves order and cart. Visible operations identify ownership rather than treating the whole aggregate snapshot as freely overwritable.

The Lifecycle of Zero Facts

Establishing zero quantum facts differs from crediting nonzero funds. Account opening prepares storage for future transfers; funding requires conserved quantum settlement.

With role and quantum facts in one database, a common local transaction establishes them. With split storage, the built-in runtime commits role records and a durable technical recovery task locally, then creates remote zero records after confirmation. Local rollback removes both. Remote unavailability or lost responses retain the task for retry. Retry creates only missing zero records, never overwrites settled values, and produces no quantum transaction journal entry.

Scenarios still call role insertion without manually splitting opening or writing business compensation. Opening is not fully successful before remote zero records exist. The current mechanism covers built-in JDBC ordinary and quantum repositories; see split zero-record recovery.

Every fact should have an owner, locator and maintenance entry. See the transaction and storage specification.

Chapter 19 — Transaction Boundaries, Execution Plans and Deployment Capabilities

“These changes must hold together” is a business requirement; resources and protocols are implementation. SPP expresses boundaries with transaction, APP selects strategy, PHY supplies real resources, and generated plans connect them.

Ordinary Updates Default to Optimistic Locking

The system retains query baselines. After scenario calculation and validation, built-in updates conditionally write against those baselines. If another request changed ordinary facts, the update reports conflict rather than overwriting them. APP defines retry or recovery.

Two requests read one profile. The first changes address; the second changes phone using stale facts. Its save must detect the changed baseline instead of restoring the old address with the whole old profile. Optimistic locking avoids holding a record lock throughout query and calculation while checking competition at write time.

This coexists with automatic locking for queries inside a transaction. Outside queries establish baselines without prelocking; inside queries lock according to property type and the transaction's modifications.

Queried Facts	Read Inside a Transaction
Ordinary participant	Automatic locking; current JDBC uses `SELECT ... FOR UPDATE`

Queried Facts	Read Inside a Transaction
Quantum facts	No ordinary read lock; quantum implementation protects settlement
Participant with ordinary and quantum facts	Ordinary facts lock if this transaction modifies them; quantum-only changes add no ordinary read lock

SPP does not explicitly write select for update, but runtime locking exists. Outside profile queries protect later saves with baselines; inside queries lock ordinary records. Locking those does not mean locking wallet quantum facts. See transaction reads.

Local Joint Commit

Petstore develop uses LOCAL. Domains share a JVM and the required database, schema and connection resources, so quantum and ordinary changes enter one local transaction. Recovery applies within that resource scope.

The same database name does not prove a common transaction connection. Separate processes do not gain LOCAL merely by accessing one database. Compilation and startup validate actual identities.

Phased Execution

Outside LOCAL, a concentrated quantum group may settle first, followed by ordinary groups according to mode. Saga restores committed phases. A later ordinary rollback cannot undo an already committed quantum change.

Plans require explicit recovery capabilities. Unsupported combinations should be rejected rather than inventing compensation. Joint success across phases differs from single-resource atomic commit.

Nested Boundaries

REQUIRED joins the current boundary; ending it does not commit separately. Payment uses it for a logical scope, not independently committing payment before order.

REQUIRES_NEW is independent and does not automatically enter the outer recovery chain. Independent account opening may remain completed after outer failure and needs its own handling. Appendix E describes example-backend propagation support.

Automatic Quantum-First Planning

Ordinary updates are built-in and contain no business query, calculation or validation. Scenarios supply complete facts through preparation and quantum results.

Ordinary saves do not affect quantum inputs or outcomes, and quantum order is exchangeable. The compiler therefore schedules quantum transactions before ordinary updates within a boundary, allowing unordered or merged execution. Serial groups arrange post-settlement calculations and validations before ready saves run concurrently. Runtime supplies context, resources, joining and failure handling. “After quantum services” describes phases; LOCAL can still commit them together at the end.

Why SPP Has No Exception Syntax

SPP omits `try/catch/throw` so scenarios describe business rules and processing while APP defines technical-failure strategy executed by runtime and PHY. Business specialists need eligibility and joint-change requirements, not database connections, timeout types or low-level exceptions. Design separates error interception from business flow; participant execution also excludes technical exceptions as SPP branch conditions.

Unsatisfied business conditions are explicit results. A shipped order may be ineligible for cancellation. Ordinary check ends its block; guard controls its enclosed steps; `breakTransction` explicitly requests reversal. Conditions and effects stay visible rather than hidden in exception classes.

Technical faults are not business facts. A database failure does not mean no order exists; payment timeout does not mean no settlement occurred. Catching these and choosing “not found” or “pay again” risks wrong decisions. APP defines confirmation, retry and recovery using execution and commit evidence, without presenting uncertainty as fact.

This also avoids coupling scenarios to providers' database and communication errors. Changing deployment should not force business rewrites. APP owns technical policies; generated Java and runtime can still use exceptions to implement them.

Separation does not remove business failure decisions. After acceptance, should unsuccessful processing leave an application pending, cancel it or refund fees? These choices alter business results and must be specified.

Business Failure Compensation

SPP handles business failures: once conditions are known not to hold, what should happen to established results? Different businesses may choose different responses.

Insufficient inventory may lead to waiting for restocking or cancellation and refund. The framework confirms reservations and payment, but cannot choose for parties. `transaction { ... } fail { ... }` expresses post-failure business operations. Technical execution and recovery occur first; the adjacent fail receives failure facts. Its saves use a new explicit transaction.

Failure	Decision Required from Business Developers
Admission rejects before changes	Reject with reason; do not refund a debit that never happened

Failure	Decision Required from Business Developers
Inventory held but later business validation rejects	Whether to cancel, releasing the actual reservation if canceled
Payment settled but fulfillment impossible	Keep payment for later handling, or cancel, refund and release holds
Application independently accepted but approval rejected	Keep for retry or mark rejected/canceled, and decide fee treatment
Goods delivered but later business cannot finish	Return, refund or another remedy; state changes do not return physical goods

These are decisions, not a universal instruction to reverse all steps. Uncommitted changes can roll back; quantum services automatically reverse original transactions without a separately written business inverse. Business still decides cancellation, release or continued fulfillment. Later partial refunds or fees are new activities, not erasing original payment history.

Compensation respects layering. Refund eligibility is SPP; timeout executing the refund is APP. SPP defines qualifications, amounts and result; APP confirms, retries and recovers. Timeout is not business rejection.

Example: Retain an Accepted Reservation and Record Rejection

Petstore saves acceptance before reserving inventory. With ten available and eleven requested, acceptance exists but reservation cannot settle. The scenario retains the original number and records rejection for insufficient stock. From the actual scenario, omitting initial checks:

```
transaction { insertPetReservationAgreement; }
prepareReservationInventory; findStockLine; identify(StockLine[]);
transaction {
    reserveRequestedInventory bounds {
        StockLine[].available: [0, null]; StockLine[].reserved: [0, null]; };
    confirmPetReservation; updatePetReservationAgreement;
} fail {
    if (ResultInfo.failure.recoveryStatus == "ROLLED_BACK" ||
ResultInfo.failure.recoveryStatus == "REVERSED") {
        findStockLine; identify(StockLine[]);
        rejectPetReservation;
        transaction { updatePetReservationAgreement; }
    }
}
```

The first transaction independently saves acceptance; failure of the later one does not delete it. Success marks RESERVED. Failure first rolls back uncommitted effects or automatically reverses the settled quantum phase, then enters fail.

`ResultInfo` is a built-in scenario participant, not an input declaration. The example acts only for ROLLED_BACK or REVERSED: confirmed rollback or recovery of committed phases. It re-queries stock, calculates rejection and saves state in a fresh transaction, outside the failed boundary.

Rejection calculation marks REJECTED and distinguishes INSUFFICIENT_STOCK from RESERVATION_NOT_COMPLETED using recovered stock and requested quantity. Ten available against eleven yields the same reservation number, insufficient-stock reason and unchanged 10 available/0 reserved. A later ordinary-save fault despite sufficient stock instead records reservation not completed.

fail does not manually add inventory back. System rollback or automatic reversal handles that; fail expresses the business choice to preserve acceptance and record final rejection.

Unknown outcomes give RECOVERY_REQUIRED, so this example does not declare safe rejection. fail may record pending verification facts, but technical failure continues to APP and is not considered recovered merely because fail ran. After successful handling, status remains FAILED with handled true, rather than converting the original transaction to success. Current support is outer transaction fail in serial scenarios; multiple input contracts use `fail(targetContract)`. See failure and system results.

Technical Failures Are Defined in APP

Update timeout may precede commit or follow commit with lost confirmation. APP defines original-request result queries, permitted retries, durable recovery tasks and operational escalation when confirmation is impossible. Runtime and PHY execute the policy.

Order-update timeout does not require SPP to catch it and cancel. APP first confirms the original update, restores a completed result or retries under the contract, and retains pending status if uncertain. Timeout alone does not trigger refund or cancellation.

Technical failures during refund or release also need APP progress, duplicate protection, confirmation and recovery. Report compensation complete only after business effects actually complete; retry must not duplicate them.

Report Actual Results

Request failure may mean no commit, partially committed phases or an unknown outcome. Settlement and recovery evidence determine actual state; an HTTP error alone does not prove rollback everywhere.

Explicit transaction semantics and deployment capabilities turn boundaries into reliable promises. Appendices D and F state detailed support.

Chapter 20 — Quantum Properties, Call Bounds and Conserved Settlement

Balances and inventory frequently face concurrent changes. “Enough when queried” is insufficient; actual settlement requires bounds and consistency. Quantum properties and services make that need explicit.

Properties and Deltas

Petstore declares wallet quantum and defines bilateral funding:

```
quantum update transferWalletFunds(
    Buyer, SettlementAccount, WalletFundingAgreement) {
    SettlementAccount.balance -= WalletFundingAgreement.amount;
    Buyer.wallet += WalletFundingAgreement.amount;
}
```

The service states the relationship; bounds specify this call's range. Limits generally belong to wallet-role properties, so scenarios query them before passing them. One transfer service can serve wallets with different limits.

Query Wallet Limits First

A normal wallet may range from zero to 10,000 US dollars, while overdraft permits -500. Such facts belong to particular wallets rather than a hard-coded universal nonnegative transfer rule.

Payment queries the source lower bound; funding queries the destination upper bound; transfers obtain both sides' relevant limits. Adding 300 to 9,800 with a 10,000 maximum violates the range.

Limit queries retrieve settlement rules; balance queries support display or preliminary judgment. The scenario passes the limits, and execution checks them against actual settlement values. An earlier balance cannot guarantee success.

Bounds Apply at Settlement

Two requests see 100 and each pays 80. Both preliminary checks can pass, but at most one settles without overdraft. The quantum implementation processes current values and checks constraints before commit.

Limits are evaluated before the call; null disables that direction's check. It does not mean unknown amount or replace a legitimate funding source. Eligibility remains explicit validation.

Conserve Actual Relationships

Payment source decrease and destination increase match. Reservation moves available to reserved; shipping reserved to sold. Each goods identity conserves separately; another product's increase cannot hide this one's loss.

Money and inventory use different dimensions, not one numerical conservation equation. They can cooperate under a scenario with separate conservation groups and an overall transaction or recovery plan.

Settlement Results and Participant Completion

The framework fills participants with settlement results. These describe the transaction's facts, not a global simultaneous snapshot across sources. Later transactions can continue changing values.

Ordinary prepared inputs remain stable as agreed; dynamic quantum facts belong to quantum execution. Business must not bypass it to save nonzero balance changes.

Fully Automatic Inverse Transactions

To undo a quantum settlement, the system locates its original record and applies opposite deltas to original participants, without a developer-written inverse or corresponding compensation transaction.

A transfer of 100 from A to B reverses 100 from B to A. This restores the transaction's effect, not entire old balances, retaining other changes. Identity, inverse execution, deduplication and technical recovery belong to the system.

Other business handling remains explicit: cancel an order, return physical goods or retain an application. None requires reimplementing quantum reversal. The next chapter distinguishes rollback, inverse transactions and compensation.

Chapter 21 — Rollback, Inverse Transactions, Reversals and Compensation

“Undo” hides distinct duties: uncommitted rollback, settled-fund reversal, new business reversal and external compensation. Distinguishing them explains failure outcomes and recovery scope.

Rollback Undoes an Unfinished Transaction

Technical rollback removes the current transaction's effects, retaining evidence and other independent committed results. Resources and protocols determine scope.

Committed Saga nodes are outside their original local transactions. The failed node can roll back its uncommitted effects; prior successful nodes need separate recovery.

Ordinary Updates Can Also Be Automatically Reversed

Built-in ordinary updates contain no calculation or query for completion. They save already prepared facts. Systems can retain before facts and resulting versions to undo writes without a reverse service recalculating old values.

An address changes from A at v8 to B at v9. Recovery can restore A only if current version still equals this write's v9, and must produce a new version. Reverting the marker to v8 could wrongly accept formerly stale requests.

If another request produced C at v10, recovery cannot overwrite it. Even if values happen to return to B, the new version still matters. Version check and restore must be one atomic conditional operation, not a check followed by unconditional write.

Quantum reversal negates a delta while retaining later deltas; ordinary recovery restores facts only while the version qualifies, without later writes. Conflict requires APP confirmation and handling, not overwriting subsequent results.

Recovering Mixed Cross-Domain Transactions

One SPP boundary may update different domains, processes and databases. It expresses joint business requirements, not a shared local database transaction. APP plans and PHY capabilities determine restoration.

Consider four updates:

Update	Effect	Domain
Q1	Buyer wallet -100, merchant funds +100	Payment roles' domains
Q2	Available stock -1, order hold +1	Inventory
U1	Save order and agreed lines	Order
U2	Save processing receipt	Receipt owner

Q1 and Q2 conserve different dimensions. Roles can belong to separate domains while configured unified quantum execution handles deltas. Ordinary repositories may be called remotely. Current non-LOCAL plans

complete the quantum group, execute ordinary phases and retain commit records. Multiple quantum requests can settle together; ordinary updates follow configured coordination.

If the quantum group does not settle, ordinary execution stops. A balance or stock bound can reject the whole group. Its resource removes uncommitted effects and U1/U2 do not proceed. A purchase cannot debit without stock or save a success receipt for no settlement. Unknown quantum commit requires confirmation, not definite rejection.

If quantum settlement succeeds but ordinary commit fails, restore both appropriately. Suppose Q1/Q2 settle, U1 executes and U2 reports a definite conflict. If U1/U2 belong to a supported XA or Seata AT ordinary scope, that coordinator first restores its writes, then the mixed plan automatically reverses the quantum group. Executing U1 is not proof it committed independently; status follows its phase.

```
Q1 + Q2 settled
  → Execute U1 and U2
  → U2 definitely fails
  → Undo unfinished ordinary transaction
  → Automatically reverse settled quantum group
  → Confirm every recovery result
```

Independently committed ordinary phases need their own recovery. Under Saga, U1 may commit before U2 fails. Roll back U2's unfinished transaction, restore U1 in reverse committed-phase order, then reverse quantum changes. U2's rollback cannot remove U1. Version-protected restoration uses U1 before facts and written version; if no later version exists it can restore without business recalculation. Otherwise retain a recovery conflict under APP policy.

Unknown phase outcomes require confirmation first. A remote U2 timeout may mean the receipt committed or never ran. Current mixed plans retain pending recovery instead of immediately reversing earlier commits. APP queries the original request, then completes or restores according to actual records. Otherwise funds and stock could be undone despite successfully saved order and receipt.

Recovery failure means recovery remains incomplete. Failed U1 restoration or lost quantum-reversal response retains original and recovery evidence. Current implementation records a failed recovery phase and still attempts others, potentially producing partial recovery. Every result must be checked; “recovery started” is not “fully undone.” APP defines later confirmation, retry and handling. Quantum inverse deduplication prevents duplicate refunds, and permitted business during recovery follows explicit policy.

Rollback concerns the uncommitted phase; quantum inverse restores committed deltas; ordinary reversal restores qualifying versions. A later bank reversal is new business, not this automatic failure recovery.

These capabilities need before facts, versions, records and backend protocols. Shared databases do not imply shared connections, and naming XA, Seata AT or Saga does not prove deployment guarantees. See environments and storage contracts.

Local Rollback and Split-Storage Recovery in the Bank Case

The bank case in Part VII, Chapter 33 provides a concrete comparison: cash reversal has already applied reverse fund movements when saving reversal postings fails.

Deployment and Failure Point	Actual Funds and Ordinary Facts	System Outcome
Shared local transaction resource; reversal postings fail to save	Funds, the original-transaction flag and reversal contract roll back together; the original remains unreversed	`ROLLED_BACK`
Separate quantum and ordinary storage; posting save fails after some ordinary stages commit	Quantum inverse transactions restore funds; ordinary flags or reversal contracts may remain	`RECOVERY_REQUIRED`, `handled=false`
Saving the failure-disposition record also fails	Preserve the original failure, actual recovery state and secondary disposition failure	Entering `fail` does not establish successful disposition

The case's split-storage repository has not yet implemented automatic recovery for committed ordinary stages. This does not change the principle of version-guarded automatic reversal, but shows that the backend protocol must actually be implemented and verified. Recording reconciliation facts is failure disposition; restoring funds and ordinary records is transaction recovery.

Tests also change an original contract's version after it has been queried, verifying that reversal cannot overwrite a later change. Even when split-storage funds have been restored, earlier ordinary stages may remain. The system preserves the pending-recovery result rather than allowing a direct retry to hide it. The case verification record documents the persisted outcomes.

Automatic Inverse Transactions Retain Later Results

Starting at 100, +50 then +20 gives 170. Reversing the first subtracts 50, leaving 120. Restoring a snapshot of 100 would erase the second activity.

The system inverses original settlement components and maintains participant identity, duplicate protection and confirmation. APP handles technical confirmation and recovery. Failure of the inverse request does not prove the inverse did not settle.

Business Reversal Is a New Activity

A bank reverses a completed transaction for business reasons by querying the original, checking eligibility and forming a new result. Original principal and interest components must be restored, not recalculated under today's policy.

It has its own transaction and failure handling. Cross-day rules, later dependencies, disbursed funds and external effects may constrain eligibility. This differs from technical rollback before original commit.

Compensation Requires Business Decisions

Consumed goods or irreversible external effects may require another remedy. Content, triggers and completion criteria must be agreed; systems execute, record and retry the defined rules.

Automatic quantum inverses and version-qualified ordinary restoration do not decide physical returns or substitute fulfillment. Database restoration cannot erase external effects, and later ordinary versions prohibit forced old-value restoration.

APP Converts Definite Technical Failures into Business Failures

Conversion requires a known outcome and agreed business meaning. A definitely rejected unique-key write may become “number already taken”; optimistic conflict may become “information changed; reconfirm.” SPP handles the result without database error types. APP defines these mappings explicitly, rather than classifying every error as one rejection.

Timeout, disconnection or lost commit confirmation can occur after commit. They indicate absence of a definite response, not business failure. APP confirms before returning success or rejection; until then, retain uncertainty and technical recovery, without triggering business compensation.

Separation therefore permits cross-layer failure information: definite, meaningful failures become business failures; uncertain ones require confirmation first.

Unknown Outcomes

Timeout can precede or follow commit. Stable request identity and result lookup confirm the original instead of starting a duplicate with a new identity.

The repository contains inverse execution and several recovery evidence paths. Complete durable coordination and compensation have specific coverage limits. Appendix F separates existing evidence from unresolved matters.

Chapter 22 — Generation, Physical Resources and Runtime Configuration

Declarative models must become executable code, interfaces, repositories and resource access. Generation performs this conversion and checks model-implementation contracts. Reading output reveals declarations' effects.

From Models to Runtime Artifacts

The compiler parses models, calls, role relationships and boundaries, combines APP/PHY, and generates Java models, services, mappings, OpenAPI, DDL and runtime catalogs according to module and configuration.

This supports Part III, Chapter 12: AI mainly maintains SPP, APP and PHY while tools deterministically generate repetitive models, interfaces and integration. Fewer free implementation choices mean fewer consistency relationships for AI to maintain manually.

Business review focuses on roles, rules, scenarios and actual results. Toolchain changes receive their own generation checks instead of reviewing all mixed code on each business change.

Business source stays in source directories; generated Java is in target. Direct changes to generated files affect only one build and may be overwritten. Maintain rules in SPP and source configuration.

Generation and Startup Checks

Generation detects undefined facts, ambiguous binding, illegal calls, unsupported propagation and contradictory resources. Startup checks actual override parameters, environments, identities, provider contracts and readiness.

Static checks do not prove current remote settlement capability, and startup does not replace concurrency or fault tests. Each supplies different evidence.

Build and Local Demonstration

Run at repository root with the documented JDK 17, Maven and dependencies:

```
mvn -pl :petstore-demo -am package -DskipTests -Dspp.environment=develop
SPP_PETSTORE_JDBC_URL='jdbc:h2:file:/tmp/spp-book-petstore' \
  ./demos/petstore/run-demo.sh web
```

The web demo defaults to 8086. This uses an independent experimental database; reusing the path retains old data. Appendix E provides variables and multiprocess commands.

Numbers and Resources

PHY selects representation and capacity; APP organizes technical mappings. SPP specifies rounding; inexact unsupported conversion must fail rather than silently alter meaning.

develop and testing can use different deployments and plans for one scenario. Compare selected environment, generated catalog and startup arguments rather than using artifacts from one to prove another.

Verify That Declarations Are Honored

A successful build establishes source-generation checks, not actual business completion. Execute scenarios and verify results. The later Petstore practice follows requirements through fund and stock movement to show the architecture as a complete system.

Part V — Separating Business and Technology

Business scenarios describe participants, calculations, settlement and failure handling. We now examine the technical capabilities surrounding them: performance, security, reliability, observability, governance and evolution.

Using different needs of one order, this part introduces caches, on-demand data, protection, idempotency, duplicate guards and governance. APP selects strategies and interception scope; PHY binds Java technical services, resources and protocols. New technical implementations can extend capabilities without technical calls in SPP. Chapter 23 introduces binding and Chapter 27 a custom implementation; each capability must fit its actual interface.

Chapter 23 — Benefits of Separating Business and Technology

Payment requires eligible parties to transfer an agreed amount and form a verifiable result. Encryption, caches, duplicate detection and database access support execution. S++ puts business in SPP, technical strategies in APP and resources and protocols in PHY, giving each a clear maintenance location.

Let Business Specialists Review Business Directly

Changing a member discount from ten to fifteen percent requires checking eligibility, percentage and quotation. Mixed database, conversion and retry code makes specialists understand technology before assessing

completeness.

SPP expresses discounts as calculations, conditions as validations and invocation as scenarios. Requirements specialists can review and model with AI. Tools supply queries, saves and integration, so review concerns business facts and rules. Part III, Chapter 12, explains how this reduces freely generated AI code.

Technical work also concentrates. Poor cache hit rates concern access policies; duplicates concern request identification and execution, not payment formulas.

Adapt One Business to Different Environments

Customer facts can come from local storage or a remote domain. Protocols and protection can change while scenarios still use Buyer with the same meaning.

Moving the customer domain to a separate application changes APP bindings and PHY endpoints and resources, requiring transaction and recovery checks. If providers honor the same contract, payment gains no HTTP conversion or connection-management code.

Replacement must preserve meaning, effects and failures, not just types. A new interface cannot treat timeout as the old “not found.” Separation makes adaptation and verification explicit.

Reuse Technical Capabilities Centrally

Orders, reservations and funding all need request identification. Separate lock, registration, result and recovery implementations can drift. A shared technical protocol with scenario-specific configuration reduces duplication and centralizes improvements.

Encryption, compression, tracing and throttling have similar value. Technical developers maintain facilities while business describes facts and results. Updated capability can serve many scenarios through configuration.

Bind Java Technical Services Through APP Interception

Capabilities need not all be built into the compiler. Technical developers can implement Java services and bind them around scenario or atomic calls. SPP remains business; APP chooses targets and stages; PHY chooses classes and parameters.

To record all shop-service durations without modifying each payment, order or query, Petstore's technical APP declares:

```
technical services { invocationTrace; }
intercept services {
    before: invocationTrace;
    after: invocationTrace;
```

```
    failure: invocationTrace;
}
```

`invocationTrace` names a capability. `intercept services` matches all generated services in this domain, including scenarios, ordinary CRUD and quantum services. `intercept scenarios` limits it to scenarios; `intercept findBuyer, reserveShopPet` selects named services. This is a technical binding, not a business call to a logging atomic service.

Its PHY specifies:

```
phy {
    technical service invocationTrace {
        implementation: "free.servpp.service.technical.InvocationTraceService";
        config logger = "spp.petstore.invocations";
    }
}
```

New implementations implement `TechnicalService` and enter the runtime classpath. APP declares name and scope; PHY binds class and configuration. Chapter 27 builds a slow-call observer.

Stage	When	Typical Use
before	Before business invocation	Technical access control and start recording
after	After normal return	Duration, validation result and completion recording
failure	When technical error propagates	Diagnostics and alerts

`after` means normal method return, not validation acceptance or outer transaction commit. A reservation rejected and handled by fail can return normally and trigger after. Recording after alone cannot certify successful payment.

The interface exposes read-only service identity, kind, trusted headers, duration, validation result and technical error, not mutable participants. It directly supports observation and identity-based control. Data rewriting, query replacement or response changes need suitable adapters. Java capability can evolve while business ownership stays explicit. See the interception protocol.

Optimization Must Honor Business Agreements

Display names may tolerate caching; payment cannot overwrite balances based on caches. Lists may return summaries, while returns need eligibility facts.

Business defines fact purposes and results; technology chooses strategies. It cannot omit checks or relax quantum bounds for speed or success rate. Freshness, recovery and outputs must honor agreements.

The next chapters discuss performance, security, reliable execution and governance. Facilities may reside in runtime, integration or adapters, without each becoming a remote microservice.

Chapter 24 — Performance Through Caching and On-Demand Data

Performance means reducing repeated access and unnecessary data as well as concurrency. SPP retains role organization; APP selects query, transfer and output strategies; PHY supplies resources.

Observe Performance Before Choosing an Optimization

APP after/failure can collect query and scenario names, durations and faults, identifying whether product access or saving dominates. No timing code enters SPP. A Java collector can integrate metrics through PHY.

Current interceptors only expose invocation metadata. They cannot modify participants or return a cached answer while skipping a query. Observation fits directly; caches, field selection and response projection belong in query, repository or output adapters.

Caches Reduce Repeated Access

Ten cart lines may reuse product information. Request caches reuse facts within a call; process caches share across requests; shared caches reduce source-domain reads across application instances.

Remote product names and categories can be cached locally. Business still calls role queries; technical cache hits complete facts without explicit “cache first, database next” in scenarios.

Keys must distinguish participant identity, loaded fields and version. Having a name does not imply having price and status. Freshness follows purpose: name display may tolerate age, while settlement needs authoritative mechanisms. Cached balances can support display or preparation, not settlement.

On-Demand Queries Reduce Read Sets

A role chain may expose many properties while quotation needs only tier, price and quantity. Compile-time use analysis can form a required set, with queries retrieving its sources.

Buyer may include name, avatar, address and tier. Discount only needs tier; ordering adds address. Role granularity remains Buyer while loaded volume decreases.

Retain identity, internal baselines and loaded-property markers. An unloaded address must not become an empty saved address. Partial loading and saving need a protocol, not merely shorter SQL.

On-Demand Responses Reduce Output Sets

Calculation input and display needs differ. Quotation may use addresses and complete pricing facts while showing only names, quantity, unit price and total.

APP can define output sets per entry: order summaries, detailed lines, or payment receipt and system result. Roles and contracts retain meaning; output adapters select allowed needed fields.

Queries determine retrieval; response selection determines disclosure. Internal sensitive facts need not reach the page. Selection must cooperate with permissions, masking and replay.

Current generated web interfaces mainly return completed input objects and result information. General projection, role-use query pruning and multilevel caches need further implementation and scope verification. These are optimization methods, not universally completed capabilities. Appendix C13 specifies conditions.

Batching and Execution Paths

Independent goods can use batch or parallel queries. Quantum services can merge requests. Ready ordinary updates can save concurrently.

Calculations and validations normally run in the scenario process, avoiding network calls for small work. Compressing large payloads saves bandwidth but adds encoding cost and should follow measurement.

Measure full latency, read/output volume, cache hits and source calls. Faster lists that cause many later detail reads may need adjusted data sets. Appendix C gives experiments.

Chapter 25 — Encryption, Fingerprints and Security Services

Correct rules also require trusted requests, protected data and traceable operations. APP organizes authentication, encryption, integrity and output control; PHY supplies protocols, keys and resources. Scenarios operate on trusted business facts.

Protect Transfer and Storage

Customer addresses may travel across domains and be stored. Integration can protect transport and select field encryption without business handling ciphertext, key choice or decryption calls.

Transport and at-rest encryption have distinct scopes and do not replace each other. Specialized facilities manage keys; PHY supplies endpoints and configuration. Rotation and upgrades affect adaptation while meaning stays stable.

Authentication and decryption may precede business; output encryption follows. Technical ordering has its own dependencies and is not arbitrarily exchangeable because business atomics can be concurrent.

Fingerprints Identify Content; Integrity Protection Verifies It

A content fingerprint is a normalized digest for detecting change. A request number first transferring 100 and later 200 is not a retry; comparing facts or fingerprints detects the conflict.

Cover relevant content, including parties, not just amount. Consistent normalization avoids treating reordered fields as changed business. Identity identifies the call; fingerprint checks its content against the original.

A plain digest does not authenticate a sender or stop an attacker replacing both content and digest. Use authenticated message checks or signatures when protection requires origin and integrity. SPP receives verified facts; integration establishes trust.

Biometric fingerprints also belong to authentication. If business explicitly requires verified identity, that qualification and result enter business rules. Means and eligibility have separate places.

Authentication, Permissions and Business Eligibility

Authentication establishes who calls, access control what entries and data they may access, and business validation whether this activity is eligible.

A logged-in buyer may query only authorized orders. Refunds additionally check ownership, delivery and deadlines. APP supplies trusted identity and protection; SPP expresses relationships and qualifications.

A submitted Buyer object is not proof of its identity, and a browser duplicate marker is not authentication. Integration verifies origin and binds trusted identity to participants before business processing.

Technical Access Control with before

An internal entry may accept only specified verified callers. A Java technical service reads trusted headers in before and rejects unauthorized calls before business starts. APP selects entries, PHY selects implementation, and SPP needs no technical identity code.

before failure prevents invocation and enters failure with the original technical error. Trusted headers must already be established by authentication; arbitrary user values cannot become credentials. Ownership and payment eligibility remain SPP checks.

Current metadata excludes addresses, amounts and message contents. Field encryption, content fingerprinting and masking need data or communication adapters. Interception can audit calls, but committed-transaction evidence must also use transaction/storage records.

Output Control and Audit

Order details may show a telephone number while lists show a masked one. APP can combine permissions, selected fields and masking, including logs and replay rather than only normal responses.

Technical audit records callers, entries, verification and traces. Business contracts record parties, amounts and processing results. Correlated identities support distinct safety and business inquiries without exposing keys or unnecessary sensitive facts.

Facilities can serve many scenarios, but algorithms, key management, authentication and permissions require implementations and scope verification. This chapter does not assume all security infrastructure is built in.

Chapter 26 — Idempotency, Duplicate Guards and Reliable Execution

Repeated clicks, communication retries and lost confirmation can deliver payment requests more than once. Reliability identifies their relationship to original business so duplicate transport does not duplicate debit. APP defines strategy; SPP retains qualifications, amount and outcome.

Idempotency Identifies the Same Request

A 100-dollar payment may settle without its response reaching the page. Retrying the same identity and content should restore the result, not debit again.

Current remote-update protocols identify caller, caller serial and target service. Identical content replays original participant facts and result; changed amount or payee under the same serial conflicts. Registration tracks running, completed and recovery-needed requests, not just a seen flag.

An order number and technical request serial differ. One order has many processing activities; a retry must retain the original call identity. New serials on every retry prevent recognizing the same payment.

Duplicate Guards Identify Repeated Operations

Transport idempotency concerns one request; UI guards also catch similar clicks with different serials. Repeated “add to cart” may be one accidental action.

APP declares identity keys, facts and a window, such as buyer, goods and quantity. After expiration, another addition can be legitimate; rejection must not be permanent.

Whether an order can be paid again is SPP business eligibility. Expired technical windows do not make it payable or imply no previous settlement. Both protections cooperate.

Current remote idempotency and UI guards are separate. Local calls do not repeat transport registration; quantum deduplication belongs to its backend. Queries do not replay through the update protocol; query caches are distinct. See call idempotency.

Optimistic Locking Protects Competition Between Different Requests

Two legitimate profile edits may share stale facts without being duplicates. Conditional updates compare query baselines and reject overwrites after new versions. Quantum services apply deltas and actual bounds.

Deduplication, versions and quantum settlement protect repeated calls, competing replacement and quantitative change respectively. One duplicate key cannot replace all three.

Confirm Outcomes Before Retrying

Timeout may mean uncommitted work or lost confirmation of committed work. APP queries original execution or retains recovery status. Repeating an entire uncertain scenario can duplicate debit or acceptance.

Confirmed full restoration with no independent stage committed may permit retry. Partial or unknown completion needs continued original recovery. Quantum inverses are automatic; ordinary restoration follows version contracts; business handling follows rules. Part IV, Chapter 21, distinguishes them.

Retries need deadlines, limits and backoff. Exhaustion does not create a compensation policy; preserve evidence and enter defined technical handling.

Interceptors Observe; Execution Protocols Confirm

Java services can correlate request identity, target and failures. Scenario entry events may include idempotent replay; if replay does not execute atomics, it produces no repeated atomic execution events.

Two entry events do not prove two debits; atomic after does not prove outer commit. Registration, journals and receipts establish actual duplication.

Current after/failure are observation stages, and failures do not replace original results/errors. There is no arbitrary retry, result mutation or independent commit callback. failure must not simply rerun payment. Execution protocols retain confirmation and reversal responsibility; interception supplies diagnostics.

Durable Records Survive Restart

Memory-only registration can forget requests after restart. Required update recovery uses durable identities and results coordinated with actual settlement.

The framework supports memory and JDBC stores with different capabilities. Unknown or partially committed records cannot expire into brand-new requests. Split zero creation also uses durable technical tasks rather than new business compensation. Central strategies keep scenarios focused on results rather than rewritten retry protocols.

Chapter 27 — Runtime Governance and Technical Evolution

Correct models still need capability under variable load, dependency failure and migration. APP organizes tracing, throttling, isolation and recovery; PHY supplies deployment and monitoring. These facilities help detect, locate and handle problems.

Traces and Metrics Locate Problems

Slow ordering may reflect product queries, stock settlement waiting or ordinary-save conflicts. “Order failed” alone identifies none.

Trace scenarios, domain calls, phases and recovery. Measure duration, errors, cache hits, version conflicts and pending requests. Distinguish settled quantum work, failed ordinary work and unconfirmed inverses.

Contracts answer what was bought, paid and processed; traces answer paths, failures and recovery progress. Correlation avoids exposing connections or threads as page business state.

Add a Slow-Call Observer in Java

Suppose calls over 200 ms need a separate record. Add a technical service without changing scenarios. This example uses current TechnicalService (maintainer reference: `../../appframe/src/main/java/free/servpp/service/technical/TechnicalService.java`) and TechnicalInvocation (maintainer reference: `../../appframe/src/main/java/free/servpp/service/technical/TechnicalInvocation.java`):

```
package example.technical;

import java.util.Map;
import java.util.logging.Logger;
import free.servpp.service.technical.TechnicalInvocation;
```

```
import free.servpp.service.technical.TechnicalService;

public final class SlowCallObserver implements TechnicalService {
    private long thresholdNanos;
    private Logger logger;

    public SlowCallObserver() {}

    @Override
    public void configure(Map<String, String> config) {
        long millis = Long.parseLong(config.getOrDefault("thresholdMillis", "200"));
        if (millis < 0) throw new IllegalArgumentException("Negative threshold");
        thresholdNanos = Math.multiplyExact(millis, 1_000_000L);
        logger = Logger.getLogger(config.getOrDefault("logger", "spp.slow"));
    }

    @Override
    public void handle(TechnicalInvocation event) {
        if (event.elapsedNanos() > thresholdNanos) {
            logger.warning("service=" + event.serviceName()
                + " phase=" + event.phase()
                + " elapsedMicros=" + event.elapsedNanos() / 1_000);
        }
    }
}
```

A public no-argument constructor is required. Registration calls configure; execution calls handle. Instances can receive concurrent calls, so do not store a request's start time or identity in shared fields. This example initializes threshold and logger once, and uses each event's data.

Put the class on the runtime classpath, then observe scenarios in APP:

```
technical services { slowCall; }
intercept scenarios {
    after: slowCall;
    failure: slowCall;
}
```

Bind implementation and parameters in PHY:

```
phy {
    technical service slowCall {
        implementation: "example.technical.SlowCallObserver";
        config thresholdMillis = "200";
        config logger = "spp.slow";
    }
}
```

The same structure integrates metrics, traces and alerts; identity-based controls or throttling can choose before. Thread-pool isolation, cancellation and message transformation still need runtime or communication adapters, not observation alone.

Interception Scope and Failure Meaning

All-service matching applies only to its declaring domain. A shop proxy and product provider are separate calls; observing provider internals requires product configuration. Declaration and phase-list order determines execution. Duplicate service-stage-technical-service matches fail compilation.

before may block invocation. after faults become diagnostics while later observers continue. failure faults attach to the original error while others continue, without replacing it. Logging failure cannot rewrite an already normally returned business result.

Method return also differs from commit. Quantum return may precede outer settlement; one scenario after cannot represent several sequential commits. Current interfaces have no independent commit callback. Execution records determine settlement and recovery.

Technical services are not registered as SPP atomics, avoiding recursion under all-service matching. Business completes activities; technical services surround calls with facilities.

Throttling and Isolation Protect Resources

A slow external catalog can occupy connections and threads until other business suffers. Limits, deadlines and isolated pools reduce propagation.

Throttling limits admission rate, isolation separates resources, and circuit breaking pauses access to persistently failing dependencies. These improve handling but cannot interpret unavailable dependencies as missing goods or guarantee whole-business success.

Timeout ending page waiting does not stop background updates. Confirmation and execution status must cooperate with response-time policy.

Degradation Must Preserve Meaning

Unavailable recommendations can be omitted and failed images replaced. Display adaptation can handle these.

Unavailable price sources cannot silently cause old-price settlement, nor payment faults mark orders paid. Agreements define optional output, acceptable cached facts and mandatory suspension. Technical methods honor business-approved outcomes.

Async notification is similar. Queues may carry notices, but an order-success promise cannot silently precede confirmed stock and funds. If business accepts now and processes later, SPP explicitly defines phases and

contract states.

Conversion and Environment Adaptation

Adapters normalize external field names, formats and number representations into agreed facts. Dates, serialization and compression can be shared.

A protocol change still supplies goods-role facts. Localization can use constants and resources. Currency conversion and rounding alter business amounts and are not mere formatting.

PHY supplies environment endpoints, storage, caches and keys. Operators configure existing capabilities; naming an unimplemented option does not make it work.

Verify Evolution

Each change needs objectives and coverage. Caches require freshness/invalidation; durable guards concurrency/restart; transaction deployments commits/inverses/unknown results; security trusted identity and output protection.

Technical tests honor capability contracts while business tests retain calculations, qualifications and agreements. Separate reviews meet at shared requirements. Petstore next connects roles, services, scenarios and integration into complete practice.

Part VI — Complete Petstore Practice

The first five parts established models, cooperation, language, consistency and technical separation. We now apply them to one buyer's complete journey through Petstore models, scenarios, APP and PHY: zero facts, funding, inventory receipt, quotation, ordering, payment, shipping and subsequent handling or failure.

Illustrative amounts and quantities can be checked step by step; snippets link to real files. Existing implementations and evidence are distinguished from proposed caching, on-demand data and full security/governance extensions. Experiments and detailed arguments remain in the appendices.

A Code-Size Comparison with Java

The comparison below uses the recorded source baseline described in Appendix F7. Later expanded formatting and English/USD presentation change physical line counts; these historical figures are not counts of the current downloadable files.

The four-domain, 34-scenario example covers membership, goods, cart, orders, payment, fulfillment, returns and reservations. Its source generated the following Java:

Content	Files	Code Lines
SPP business models, services and scenarios	63	999
Business enums	2	18
APP and PHY configuration	8	331
Maintained S++ source total	73	1,348
Tool-generated Java	962	100,151

Counts exclude blank lines and comments but include brace-only lines. About one thousand business lines plus three hundred configuration lines expand into about one hundred thousand Java lines, including models, request/response types, interfaces, implementations and integration. Business specialists mainly maintain SPP; tools deterministically produce expanded models, calls and persistence.

A rough Spring Boot + MyBatis + Lombok estimate is:

Java Scope	Maintained Code and Configuration
Conventional monolith for 34 scenarios using local transactions	About 6,000–10,000 lines
Including this example's domain calls, conserved transactions, durable idempotency and recovery responsibilities	About 10,000–20,000 lines

Java, SQL/Mapper and necessary configuration are counted together, as with S++ source plus configuration. Dependencies, tests, pages and generated accessors are excluded. Existing frameworks and components are assumed to handle repetition; developers need not rewrite generated output line by line.

The rough comparison is thus about 1,350 maintained S++ lines versus ten to twenty thousand conventional Java-project lines. Java ranges are responsibility-based estimates, not independently measured implementations. One hundred thousand is actual generated volume, a different quantity. Appendix F7 gives methods and assumptions.

Connecting to the First Five Parts

Practice	Principles Applied
Identify roles, contracts and responsibilities	Part I completeness and polymorphism; Part III models and services
Arrange queries, calculations, validations and saves	Part II dependencies and concurrency; Part III scenarios
Settle money/inventory and handle failure	Part I quantum services; Part IV transactions, conservation and recovery
Configure storage, requests, security, performance and deployment	Part III layers; Part IV guarantees; Part V technical capabilities

Check objective completeness, ready inputs, consistent settlement and storage, and whether technical strategies honor them.

Chapter 28 — Identifying Roles, Behavior and Services from Requirements

The earlier parts now meet in one shop: a VIP buyer funds 500 US dollars, buys two goods at 60 each, pays and receives shipment, then examines cancellation, returns and failed reservations. These are illustrative inputs, not fixed demo seeds.

First establish objectives, then SPP expression, then APP/PHY integration and deployment. Meaning, source responsibilities and results can be checked together.

Describe Complete Activities First

“Buyer purchases goods” is too broad. Expand it:

Activity	Participants	Completion Result
Funding	Buyer, settlement source, funding contract	Wallet increases; source decreases equally
Ordering	Buyer, cart, goods, stock, order	Terms saved, stock reserved, cart maintained
Payment	Buyer, merchant, order, payment contract	Funds transferred, order paid, receipt formed
Shipping	Buyer, order, inventory, shipment contract	Reserved becomes sold; shipping recorded

Activity	Participants	Completion Result
Return request	Buyer, order, request contract	Reason and processing state recorded
Return approval	Buyer, merchant, order, inventory, refund contract	Funds returned, stock replenished, result saved

Payment deserves its own scenario because wallet debit is only one part. Payee, eligibility, amount, status and receipt determine completeness. Starting from Buyer methods can scatter cooperation; starting from payment exposes participant duties. This is the emphasis of Part I, Chapters 1–2.

Separate Entities, Roles and Contracts

Person belongs to customer and Product to product. Buyer, Merchant and Goods express shop identities; inventory owns StockLine and supply. The shop does not acquire ownership of all customer/product facts.

Person plays Buyer; Product plays Goods; buyer and goods jointly play CartLine, owning relationship quantity. Goods forms OrderLine with agreed name, quantity, price and amount. ShopOrder is an ordering contract; Receipt an evidence entity.

Current name differs from agreed name. “Pet Food A” later becoming “Pet Food A — New Packaging” must not change the old order. APP uses `current(Goods.name)` and `snapshot(OrderLine.name, OrderLine.unitPrice)`, developed in Part VI, Chapter 32.

Membership and order numbers serve customers, not merely tables. `business` states that use; keys, versions and duplicate identifiers belong to technology. Part III, Chapters 13–14, now have concrete applications.

Classify Scenario Responsibilities

Responsibility	Petstore Example	Work
Query	findBuyer, findMerchant, findShopOrder	Complete existing facts
Validation	checkOrderOwner, checkPaymentEligibility	Check ownership and qualification
Calculation	preparePayment, preparePaymentAgreementReceipt	Prepare amount, state and receipt
Quantum update	payOrderFunds	Transfer wallet funds to merchant
Ordinary update	updateShopOrder, insertReceipt	Save prepared facts

Systems supply queries and ordinary saves; developers write calculations, validations and scenarios. Updates do not retrieve amounts or calculate discounts internally. Visible preparation concentrates rules in readable flows while persistence can be implemented separately.

What Polymorphism Reuses

Quotation reads Buyer tier, Goods price and CartLine quantity, without knowing the original database, supplier or external system. Another legal Goods player supplying those facts can reuse calculation and validation.

Meaning still matters: having a `price` field does not authorize arbitrary substitution for sales goods. Existing Petstore demonstrates concrete chains; new players need models, mappings and verification, not merely matching fields.

Follow Both Business and Technical Source

Read requirements, then the payment scenario and its named calculations/checks. Ask what each uses, produces and waits for.

Then read APP and PHY for storage, guards, transactions and resources. Functional directories organize source without changing domain identity or requiring a particular deployment machine.

Next establish participants and resource sources, then follow one set of amounts and quantities.

Chapter 29 — Zero Opening, Funding and Inventory Receipt

Purchasing needs buyer, store, goods, wallet and stock. Establishing participation is separate from moving money or goods: opening creates no gifted balance or stock. Every later nonzero change has a source.

Opening Prepares Identity and Zero Facts

New wallets, available/reserved/sold stock and sources begin at zero. Creation generates no nonzero transaction journal, and retries must not reset funded balances.

Business establishes identities; technology prepares records and quantum addresses. Durable split-storage tasks confirm and recover zero creation without per-scenario database retries. Part IV, Chapter 18, and Part V, Chapter 26, explain lifecycle and persistence.

After opening but before funding, the buyer exists with zero balance: an interpretable state.

Funding Shows a Conserved Pair

Funding 500 increases Buyer.wallet by 500 and decreases SettlementAccount.balance by 500. From zero, results are 500 and -500, still summing to zero.

The negative source is a counterpart within the demo boundary. Real integration must confirm funding source, payment and settlement eligibility; this does not permit unconditional gifts.

Funding source states:

```
quantum update transferWalletFunds(Buyer, SettlementAccount, WalletFundingAgreement) {
    SettlementAccount.balance -= WalletFundingAgreement.amount;
    Buyer.wallet += WalletFundingAgreement.amount;
}
```

The minimum responsibility is this pair's complete change. Splitting into unrelated remote updates loses the meaning of one funding activity.

Query Facts, Check Actual Bounds at Settlement

From the same source:

```
checkBuyerCredential; checkFundingAmount; prepareWalletFunding;
parallel { findBuyer; findSettlementAccount; }
identify(Buyer); identify(SettlementAccount);
transaction {
    transferWalletFunds bounds {
        SettlementAccount.balance: [null, null];
        Buyer.wallet: [0, null];
    };
}
```

Check inputs, prepare relationships, query buyer and source. Inputs are ready and independent, permitting concurrency. The transaction handles prepared transfer without looking up the amount inside the update.

Queried balances do not replace actual bounds: other funding or payment can intervene. This example uses a zero wallet lower bound and unlimited source bounds. Role-owned limits, if required, must be queried and passed to the quantum call.

Inventory Receipt Uses the Same Structure

Receiving ten decreases StockSupply.balance by ten and increases StockLine.available by ten. The properties differ from money but still need source, destination, quantity and settlement bounds.

The receipt scenario confirms goods, prepares stock/source, queries both concurrently and calls `receiveInventory`, whose facade binds to inventory quantum definitions. No database address or protocol enters business calls.

Similar structure does not require a generic action. Funding and stock receipt have different units and qualifications; distinct names preserve meaning.

Record a Checkable Baseline

Fact	After Opening	After Funding and Receipt
Buyer wallet	0	500
Settlement source	0	-500
Available / reserved / sold	0 / 0 / 0	10 / 0 / 0
Supply source	0	-10

Check negative-amount rejection, absent-identity nonsettlement and original-request replay without another credit. UI guards and transport idempotency do not replace positive-amount rules. Part VI, Chapter 32, gives configuration.

Chapter 30 — Quotation, Ordering, Payment and Shipping

Continue with wallet 500 and available stock ten. A VIP buys two at 60. Distinct quotation, ordering, payment and shipping activities organize different service categories.

Quotation Forms Candidate Terms

Subtotal 120, discount 12, discounted amount 108 qualifies for free shipping, so total is 108. See calculations.

Queried prices, quantities and tier produce a candidate, not persistent settlement. Lines retain names, quantities, unit prices and amounts; the order retains discount, shipping and total. Numerical price is descriptive quotation input, not automatically a quantum property.

Later price changes leave old agreed lines unchanged. Business requires historical terms; APP snapshots implement saving and querying them.

Ordering Connects Terms and Reservation

The order scenario has four dependency phases:

1. Prepare selection and query buyer/catalog concurrently.
2. Locate and query cart, validate it and calculate quotation.
3. Prepare inventory identities, query and validate reservation, prepare order/cart changes.
4. Save lines, order, cart and receipt with inventory reservation in the transaction.

Initial queries are independent. Stock query needs goods relationships prepared from the order, so cannot arbitrarily join the first parallel group.

Two units move from available to reserved: 8/2/0. Supply stays -10 and this goods total zero. Preliminary availability does not prevent competing orders, so actual bounds remain mandatory.

The compiler schedules quantum and ordinary work by preparation dependencies. `preparePurchaseReceipt` is inside the current transaction; not every statement can execute as a simultaneous save. Calculation precedes dependent receipt insertion. Built-in saves do no hidden completion.

Payment Makes Amount and Parties Explicit

Payment reads the formed order instead of cart or recalculating discounts. It verifies ownership and eligibility, prepares 108 and receipt, then transfers and saves.

From payment:

```
transaction {
    transaction REQUIRED {
        payOrderFunds bounds {
            Buyer.wallet: [0, null]; Merchant.cash: [0, null];
        };
    }
    updateShopOrder;
    insertReceipt(PaymentAgreement.receipt);
}
```

Wallet becomes 392 and merchant gains 108. Quantum services conserve funds; ordinary services save order and receipt. Their ready inputs permit parallel saving under supporting configuration.

Inner REQUIRED joins the outer logical boundary, not an independent settlement. APP/PHY determine joint physical commit or phases and recovery. SPP alone does not prove two databases share a transaction.

Shipping Uses Another Inventory Pair

Shipping queries buyer/order, prepares and queries inventory, checks status and reserved quantity, then prepares agreement and receipt.

Two move reserved to sold: available 8, reserved 0, sold 2. Ordinary saves retain state, tracking and receipt. Payment conserves money; shipping conserves quantities, with different completion conditions.

Check Each Stage

Stage	Wallet	Merchant Delta	Available / Reserved / Sold	Ordinary Facts
Ordered	500	0	8 / 2 / 0	Lines, total 108, unpaid
Paid	392	+108	8 / 2 / 0	Paid state, receipt
Shipped	392	+108	8 / 0 / 2	Shipping state, tracking, receipt

These expected illustrative results do not replace runtime evidence. Unknown payment cannot be shown as definitely failed or successful; APP confirms settlement and saves. Next distinguish recovery from subsequent business.

Chapter 31 — Cancellation, Returns, Reservations and Failure Handling

Completed order stages determine subsequent services. Cancellation and return are new activities; rollback and inverse execution restore the original failed activity. Both can reverse changes but have different meanings.

Follow Established Facts

These are alternative branches, not consecutive steps on one order:

Starting Point	Activity	Money	Inventory
Ordered, unpaid	Cancel unpaid order	None	Two reserved become available
Paid, unshipped	Cancel paid order	Refund 108	Two reserved become available
Delivered, return eligible	Request return	None	None; record request/reason
Request qualifies for approval	Approve return	Refund agreed amount	Corresponding sold units become available

Return request needs buyer, original order and request. Approval adds funds and stock. Policies requiring physical receipt first need that fact and eligibility; demo rules are not universal.

Use confirmed amounts and quantities, not old wallet/stock snapshots that erase later business.

Business Refund Versus Automatic Inverse

Canceling a successful payment is new refund business with eligibility, contract and receipt. Undoing a payment that failed during execution restores its original effects; quantum inverses need no separate business service.

Ordinary restoration protects later versions under Part IV, Chapter 21. The current implementation still requires corresponding compensation capability; simple save logic is not proof all deployments already provide automatic conditional inverse updates.

If quantum funds settle but cross-resource order save fails, APP recovers committed phases. Full confirmed recovery permits declaring reversal; partial failure remains pending. SPP does not take over remote result queries and database retries.

Reservation Intentionally Retains Acceptance

The reservation scenario first saves acceptance and number, then reserves and confirms in a second transaction. Competition may cause the second to fail while acceptance survives.

With ten available and eleven requested, quantum bounds reject reservation. After confirmed rollback or inverse execution, the scenario records rejection while keeping the number for queries.

The second boundary is:

```
transaction {
    reserveRequestedInventory bounds {
        StockLine[].available: [0, null];
        StockLine[].reserved: [0, null];
    };
    confirmPetReservation; updatePetReservationAgreement;
} fail {
    if (ResultInfo.failure.recoveryStatus == "ROLLED_BACK" ||
        ResultInfo.failure.recoveryStatus == "REVERSED") {
        findStockLine; identify(StockLine[]);
        rejectPetReservation;
        transaction { updatePetReservationAgreement; }
    }
}
```

The fresh stock query supports reason selection: insufficient current stock versus another incomplete reservation. Model and calculation define it. Business does not catch database exceptions; it handles accepted applications after recovery confirmation.

Three Different Failure Paths

Situation	Facts	Handling
Invalid quantity	No settlement	Validation rejects before stock transaction
Reservation fails; recovery confirmed	Acceptance retained, reservation undone	fail saves business rejection and reason
Reservation/recovery unknown	Resource changes unconfirmed	APP confirms and recovers; no definite rejection

Definite meaningful technical failures can reach business handling. Timeout is not automatically insufficient stock and must not make fail reserve again. The guard prevents that error.

If the rejection save fails, do not pretend it persisted. Preserve evidence and continue confirmation. ResultInfo describes this execution; stored status and reason describe the application.

Verify Completion with Faults

Check insufficient stock, ordinary-save failure after quantum changes, and unknown results. The first two require stock, application state and reason checks; unknown requires recovery records and no repeated reservation.

See failure evidence. LOCAL tests alone do not prove every real cross-domain recovery. Check resources and evidence together next.

Chapter 32 — Technical Configuration, Deployment and Integrated Verification

The preceding chapters expressed participants, quotation, settlement and failure. Now inspect existing configuration, extensions and deployment verification. Part V separation should become concrete files and results.

Put the Layers Together

Layer	Payment Content	Files
SPP	Eligibility, amount, transfer, order and receipt	Scenario, models, calculations and validations

Layer	Payment Content	Files
APP	Storage, request policy, bindings, transactions and environments	petshop.app
PHY	Database, quantum storage, routing, timeout and adapters	petshop.phy

Business reviews parties, amount, eligibility and handling; developers review storage/calls; operators resources. AI helps each, but understanding payment does not require reviewing all generated Java and drivers. Runtime evidence still verifies guarantees.

Snippets come from APP and PHY.

Retain Agreed Facts Through Mapping

```
table SHOP_ORDER_LINES(OrderLine) {
    primarykey(OrderLine.lineNumber);
    auto OrderLine.lineNumber prefix "ORDER-LINE";
    foreignkey(OrderLine.goods) references Goods;
    snapshot(OrderLine.name, OrderLine.unitPrice);
}
```

Goods name uses `current(Goods.name)`. Save an order, change product name and selling price, then query goods and old order. Current facts change, historical name/price remain. This tests Part III mapping and Part IV ownership together.

Protect Repeated Processing

Global and payment-entry declarations occur in requests and scope local:

```
requests { idempotency retention 3 days; duplicateWindow 5 seconds; }
```

```
duplicate key(Buyer.membershipNumber, ShopOrder.orderNumber)
    facts(ShopOrder.orderNumber) payShopOrder;
```

PHY uses `config requestIdempotencyAdapter = "jdbc";`. UI guards handle quick repeated actions; remote idempotency handles repeated transmission. SPP still checks eligibility.

Verify no second payment for repeated clicks, original results for same-identity/content remote replay, and conflict for changed content. LOCAL calls do not register remote transport requests and cannot substitute for its tests. See protocol.

Check actual durable data sources and restart records. Three-day retention does not allow unknown requests to become new payments when time expires; unresolved recovery needs explicit treatment.

Compare Concentrated and Split Deployment

develop deploys four domains in one process:

```
transactions {
    mode: LOCAL;
    resource: shopRecordsResource;
    ordinaryUpdates: PARALLEL;
}
```

PHY binds ordinary and quantum resources locally. Ready independent ordinary saves run concurrently; unprepared steps retain scenario dependencies.

testing separates customer and product processes while inventory and shop share a process. Ordinary and quantum databases are separate; PHY supplies routes/timeouts, APP chooses unified quantum storage and recovery for ordinary resources.

Domains stay the same while paths/resources change. LOCAL rollback is not cross-resource rollback. Quantum-first settlement followed by save failure requires phase confirmation and recovery, not relaxation of consistency.

Web, restart/conservation and failure evidence is in Appendix F. Recorded paths are neither all-fault proof nor a performance benchmark.

Apply Performance Methods to Queries

Goods browsing can cache names, descriptions and display facts. Property-use analysis can limit quotation to pricing/eligibility and listings to display, avoiding complete recursive loads.

Order lists need number, state and total; detail needs lines. Measure on-demand reads and responses separately, and never save partial objects as complete snapshots.

These proposals from Part V, Chapter 24, are not proven complete multilevel caches, compiler pruning or generic output projection in this example. Add keys, invalidation, permissions and unloaded-field save contracts, not just fewer scenario queries.

Display caches for wallets and availability do not replace actual payment/reservation bounds. “Two cached items” cannot guarantee buying two.

Extend the Existing Logging Example

Technical APP binds invocationTrace to shop services with intercept services, while PHY supplies generic Java logging. SPP has no added logging calls.

Business reviews payment, status and rejection reasons; technology reviews events. A fail-handled reservation normally returns AFTER even though reservation failed. Logs omit customer/order contents.

Part V, Chapter 27, shows custom Java metrics, alerts and controls through APP/PHY. Query caches and output masking need data adapters. See technical example.

Apply Security and Governance to Entries

testing has an interdomain token, not a complete authentication, authorization and data-protection system. Production work includes:

Capability	Location	Verification
Authentication/access control	Page and domain entries	Trusted caller; no impersonating another buyer
Encryption/masking	Transfer, addresses, logs and output	No unauthorized sensitive disclosure
Content fingerprints	Request registration	Changed content under same identity detected
Tracing/metrics	Query, settlement, save and recovery	Locate delays and establish phase results
Throttling/isolation/breakers	Browse/update entries and remote resources	Contain faults while confirming accepted updates

These are design tasks, not a claim all exist. Content fingerprints and ownership checks do not authenticate callers, as Part V, Chapter 25, explains.

Recommendations/images may degrade; payment cannot falsely report success or rejection after timeout. Correlate original requests, journals, ordinary saves and recovery, without SPP network catches.

Produce an Integrated Record

Check Chapters 29–30 numeric tables, then cancellation, return or reservation branches. Verify states, receipts, conservation, duplicates, versions, insufficient stock and unknown outcomes.

Keep performance comparison focused: serial versus parallel independent queries under identical business and resources, recording latency, overlap, throughput, errors and use. Add separate cache/projection comparisons if implemented. Appendix C holds detailed methodology.

Record source version, APP strategy, PHY resources and coverage. Business correctness, honored configuration and confirmable deployed results together complete the practice.

Part VII — Advanced Cases and Architectural Methods

Bank deposits and reversals illustrate complex rules, followed by system migration, design review and the expression of complex requirements. We distinguish role-oriented modeling, commercial service architecture and current language implementation, using a hash table to show how simple concepts form complex algorithms. The cases yield reusable methods for participating identities, behavior, ownership and execution evidence.

Chapter 33 — Bank Deposits, Repayment Allocation and Reversal

A bank's deposit business includes admission checks, authorization, repayment allocation, interest accrual, accounting and reversal. It provides a useful test of whether role-oriented service architecture can express complex rules while keeping business and technology separate.

This chapter uses the repository's bank deposit case study. Business logic is expressed in SPP, APP and PHY; tools generate the production Java code. Tests only prepare inputs, invoke generated services, assert outcomes and inject technical faults. Business rules and demonstration configuration have distinct sources; they do not represent every policy of the original bank system.

Distinct Participating Facts

Accounts, debts, product policies, business dates, authorization evidence, funding sources, processing plans and original transactions all participate. Account balance, interest-bearing amount, interest owed, principal owed and overdraft exposure have different meanings. Their common representation as amounts does not justify combining them into one field.

Front-office cash acceptance and core calculations may also have different amount conditions. Requiring a positive amount at the front office does not justify deleting a zero-amount repayment path in the core. Service contracts must preserve these differences between calling layers.

The current case separates accounts, eligibility, calculations, fees, accruals, postings, deposits and reversals into capability directories. Each scenario has its own file and directly organizes queries, checks, calculations and updates. A single umbrella scenario does not hide every step. The source navigation identifies these capabilities.

Repayment Allocation and Actual Fund Destinations

Normal debt and impaired debt can have different principal and interest repayment priorities. Business calculations first produce explicit components; the scenario then organizes account changes and postings. Eligibility and authorization checks remain visible in the scenario, while APP and PHY connect queries and storage.

Suppose a cash deposit of 100 US dollars repays 10 US dollars of interest and increases the customer's balance by 90 US dollars. The actual fund movements are:

```
Cash settlement account       -100
Customer account               +90
Bank interest-income account   +10
Total fund movement              0

Interest owed separately falls by 10: this is a debt-settlement fact.
```

Treating a ten-dollar reduction in interest owed as the destination of funds is incomplete. Reducing debt explains how much less the customer owes; it does not explain where the money actually received by the bank is held. The case therefore explicitly introduces three internal accounts: interest income, collection-interest income and principal recovery. Calculations determine the components; quantum settlement updates actual recipients; the contract records the actual accounts and amounts; postings provide corresponding accounting evidence.

Debt and exposure continue to follow the case's quantum-property conventions, but must not be counted again in the fund-conservation group above. Fund conservation and changes in debt express different facts. APP specifies the current fund-conservation group; the runtime still records actual changes, bounds and inverse-transaction information for the other quantum properties.

Components need sufficient precision. Three-decimal interest can follow the agreed business mathematics; physical columns must not silently truncate it. Any required rounding must be explicit business policy. Demonstration parameters must also be distinguished from rules traced to original sources.

Prepare Through Queries and Calculations, Then Settle Within the Transaction

The cash-deposit scenario first queries the customer account, eligibility, funding counterparties and accrual facts in parallel. It then checks ownership, eligibility and authorization, and calculates fees, repayment components, postings and accruals. Updates that must hold together belong within an explicit transaction boundary.

Opening an account establishes only a zero balance; depositing cash creates actual fund movements. The bank's internal recipient accounts must also be opened first. A first interest repayment cannot create nonzero funds from nothing.

Within the transaction, quantum settlement comes first. Subsequent calculations of accrual and processing information prepare facts for ordinary saves, which persist account metadata, the deposit contract and postings.

These calculations have dependencies preserved by the serial flow; parallel ordinary updates remain subject to their prerequisites. The scenario declares the complete business boundary, while APP and PHY implement it using actual resources.

Authorization Checks and Resuming After Authorization

Pending authorization is a business outcome, not a generic technical exception. The current case supports eligibility and authorization assessment. Persisting pending applications, choosing the authorization domain and resuming across processes still have boundaries to resolve.

A complete resumption flow should requery and recheck potentially changed eligibility after authorization arrives, then execute against current facts. Existing authorization checks do not constitute that complete workflow. The case's known issues records the distinction.

Reversal Driven by the Original Transaction

The cash-reversal scenario queries the original contract, actual postings and current account facts. It checks original ownership, recipient accounts, reversal eligibility and posting evidence, then derives reverse movements from the original settled components. Changed policies or loan status must not cause reallocation of the original amount.

If 100 US dollars is deposited and another five US dollars arrives later, reversing the first deposit must preserve the later five US dollars. Reversal must not restore the entire account to its old snapshot. Similarly, the original agreement records the fee and internal recipient accounts; current rates or account routing must not select them again.

The contract preserves a snapshot of settled postings. Before reversal, actual postings are compared against that evidence. Corrupted accounts, amounts or records must cause explicit rejection; balanced totals alone do not justify proceeding. Cross-day handling, Mode2, accruals and current eligibility also have their own checks and calculations.

A separate read-only reversal preview runs the same preparatory queries, checks and candidate calculations without settlement or persistence. Displaying eight candidate posting lines does not mean accounting facts have changed. Formal reversal must check current facts again.

Recovery from the Same Failure Under Different Deployments

In develop, related updates share a local transaction resource. If saving reversal postings fails, quantum and ordinary changes roll back together. The original transaction remains unreversed and can be retried after the outcome is confirmed.

In testing, quantum storage and ordinary records are separate. After quantum settlement, some ordinary Saga stages may already have committed. A later posting-save failure can trigger an inverse transaction for quantum funds, while already-committed ordinary facts may remain. The current case explicitly preserves `RECOVERY_REQUIRED` and `handled=false`. Restoring funds is not proof that the whole business activity has been undone.

This demonstrates the implementation boundary discussed in Part IV, Chapter 21. The architectural possibility of version-guarded automatic reversal for ordinary updates does not mean every repository and deployment already supports it. The case's `fail` block saves a failure-disposition record. It records that this transaction's fund movement was not completed only after the system confirms `ROLLED_BACK` or `REVERSED`; a reconciliation record cannot replace actual recovery.

The disposition record preserves the contract and system failure information as they stood at the time. Later queries must still show the original cause, phase and recovery status, without current balances or associations overwriting them. If saving that record also fails, the original transaction failure and actual recovery state must remain available.

Business Expression in Relation to Source Size

A source count on October 14, 2026 identified nine COBOL programs explicitly located by file or line references in the case documentation. They directly reference 135 COPYBOOKs; following nested references yields 161 distinct COPYBOOKs.

Content	Files	Physical Lines, Including Blanks and Comments	Lines Excluding Blanks and Comments
COBOL programs directly located for analysis	9	14,156	11,410
Corresponding COPYBOOKs, including nested references	161	23,488	11,179
Original-source total	170	37,644	22,589
Current SPP business source and enums	35	1,043	979
Current APP and PHY configuration	4	86	86
Current S++ source total	39	1,129	1,065

Expanding the scope to all related programs mentioned in the documentation gives 22 COBOL programs and 192 distinct COPYBOOKs: 45,587 physical lines, or 28,358 excluding blanks and comments. This broader scope includes dispatch, data access, accounting configuration and branches that have not been migrated.

These figures describe the maintenance scale of the business expression and original implementation, but their coverage differs. The original count includes entire source files; the case migrates selected business paths. Current S++ figures exclude tests, web pages, generated Java and shared frameworks. They therefore do not establish that more than thirty thousand lines of COBOL have been fully and equivalently replaced by roughly one thousand lines of S++, nor do they establish a productivity multiplier.

S++ concentrates selected rules in business models, calculations, checks and scenarios, leaving repetitive access and execution facilities to tools. Business expression becomes more focused, while tools and frameworks still implement the system. Appendix F9 provides detailed counts, source inventories and reproduction instructions.

From Web Operations to Verification Evidence

The Chinese-language deposit workbench supports account opening, cash deposits, reversal previews, formal reversals and accounting queries. The page submits participants and displays results; it does not calculate fees or repayment allocation. Communication loss is shown as an unconfirmed outcome, without automatically resubmitting an update or presenting input balances and candidate postings as settled facts.

Verification includes real JDBC storage, rule vectors, failures in two environments, concurrent contention, request idempotency, independent-process restarts and browser flows. See the latest verification record and Appendix F for configurations, batches and limits. Outstanding ordinary Saga recovery, complete authorization resumption and automatic resolution of unknown settlement outcomes remain separately documented in the completion audit.

Complex business becomes manageable by expressing rules, participating facts, processing plans, settlement and recovery separately, then connecting them through scenarios. Existing evidence supports defined paths. Original COBOL execution and bank-confirmed results remain distinct evidence levels; test counts alone cannot establish business equivalence.

Chapter 34 — Migrating an Existing Object-Oriented System

Start with one complete activity. Understand promised results, then identify participants, roles and behavior for explicit comparison with the old system.

Choose a Verifiable Business Line

Select clear inputs, durable results and failure rules, such as funding or payment. Preserve normal, rejection, duplicate and competition cases. Existing databases/interfaces can remain technical infrastructure.

The migration unit needs completion criteria. Moving a formula without actual settlement is calculation migration, not replacement of a transaction.

Identify Participating Identities from Classes

Read requirements and calls to separate User/Product duties by scenario. Identify buyer, payer, owner and fulfillment roles, players, borrowed and owned facts.

DTO fields are clues, not ready-made Roles. Reassess use, ownership and lifecycle; move technical keys and versions to application configuration.

Make Hidden Cooperation Visible

Identify access, qualification, calculation and persistence and establish atomic contracts. Place whole-activity order and boundaries in scenarios for review.

Pure calculation can migrate first with equal-input comparison, then facts, identity and settlement. Use independent expected results or historical evidence; copying a formula twice is not a correctness proof.

Control Deployment

Initially co-locate new services and existing adapters, limiting simultaneous changes to models, protocols and resources. Later split according to capacity and team ownership.

Data migration needs its own design for changed role, detail and snapshot structures. Decoding old records does not prove correct identity/ownership. Define authoritative writers during transition to avoid conflicting double writes.

Advance with Evidence

Retain versions, examples, durable outcomes and faults. Generation, local calculation, actual settlement and cross-process recovery represent different progress.

Completion returns to acceptance conditions: clear model, correct behavior, resources and operational recovery. Then new boundaries become maintainable system structure.

Chapter 35 — Architecture Review and Capability Boundaries

Review meaning, composition and guarantees. Declarations help discover errors, but designers still judge whether concepts are real, rules complete and implementation honors contracts.

Modeling Method and Architectural Approach Are Two Levels

“Role-oriented service architecture” contains two levels: role orientation is a modeling method; service architecture is an approach to organizing a system. For comparison, a service architecture based on object modeling may be called “object-oriented service architecture.” These names let us discuss modeling expressiveness separately from architectural scope. Object orientation can also support other architectures.

The service architecture in this book targets commercial business services. Purchasing, payment, reservations and shipping have explicit participating identities, processing rules and completion criteria, making them suitable for roles, services and scenarios. S++ delegates common queries, updates, conserved settlement and technical integration to tools, allowing business developers to express an activity with few declarations. This brevity comes from recurring domain structures and tool support; complex rules still need to be specified.

Fluid simulation, celestial mechanics and numerical solvers may instead focus on equations, discretization and error. Organizing those problems entirely as commercial services is usually inconvenient. A few business declarations cannot be expected to replace numerical algorithms. Task submission, resource allocation and result delivery in a scientific-computing system can use service architecture, while its solvers use expressions appropriate to their own domain.

When asking whether the architecture can meet requirements of arbitrary complexity, first distinguish a modeling method, a domain architecture and the capabilities already supported by the current language and tools. These are three separate questions.

Simple Concepts Can Also Build Complex Role Models

Object orientation starts with entities: many stateful objects receive actions, and their interactions describe a complete process. Role orientation starts with behavior: many activities identify their participating roles, and compositions of behavior describe a complete process. A scenario can be understood here as behavior in the broad sense, organizing several behaviors toward a larger objective.

Both approaches express state, change and composition; their starting points differ. Role orientation does not remove entities or data structures. It understands them through the behaviors in which they participate. A complex process can be decomposed into subbehaviors, and participants into simpler structures. Modeling can continue as long as each level has explicit state, rules and composition relationships.

A chess game, for example, can begin with board and piece objects that provide move methods. Alternatively, it can begin with moving behavior and identify the player, selected piece and board as participating roles, then organize alternating moves, position evaluation and game termination. The board still holds the position and pieces still have locations; behavior becomes the organizing thread.

Role-oriented modeling is therefore not inherently limited to commercial activities. For processes describable by state and computable rules, both approaches can extend through composition. Expressing a process, expressing it concisely and executing it efficiently are different questions. Appendix A8–A9 gives a constructive argument and its conditions.

Comparing the Starting Points with a Hash Table

A hash table shows why simple foundational concepts can compose into complex structures and algorithms.

Object orientation typically begins with a hash-table object. It holds a sparse storage table and offers `put(key, value)` and `get(key)` methods. The methods calculate positions, handle collisions, save records and find them. We first see what a hash table is and what it can do.

Role orientation can begin with “hash storage” behavior and identify three participating roles: a key used for lookup, a value to store and a sparse table providing storage. A particular string or identifier can play the key, a record the value, and a concrete structure the sparse table. The behavior explains their cooperation:

```
Hash storage (key, value, sparse table)
1. Calculate a candidate position from the key and table capacity.
2. Check whether that position already contains the same key.
3. If so, replace its associated value.
4. If another key occupies it, continue under the collision policy.
5. At an available position, save the key and value.
```

This is a modeling illustration, not executable current SPP syntax. Collision handling could use open addressing. Resizing can introduce behaviors for allocating a new table, migrating records and switching storage. Complexity remains, carried by explicit cooperating behaviors.

“Hash lookup” takes a key and sparse table, follows the same hashing and collision rules, and returns the found value or a not-found result. Storage and lookup must share these rules, or a successfully stored value might never be found.

For example, with capacity eight, let an integer key's initial position be its remainder on division by eight. Key 10 first checks position 2; key 18 also checks position 2. If key 10 occupies that position, a policy that searches forward for a free slot stores key 18 at position 3. Looking up 18 must likewise start at 2, compare the key and continue to 3. Object methods and role behaviors can both express this process.

Key and value are algorithmic participating roles here. They need not be declared as SPP business Roles, nor must every probe become a remote service. This example concerns role orientation as a foundational modeling method. Part III's business roles, atomic services and scenarios are its domain-specific constraints and applications in commercial service systems.

Reusing Object Implementations for Complex Algorithms

If needed, a general-purpose language could be built around role orientation, providing further expressions for data structures, algorithm composition and execution. Current S++ focuses on commercial business services. Demonstrating the modeling method does not require reimplementing every general computing facility.

Object orientation has already produced extensive data structures and algorithms. Role modeling can reuse these achievements: the business layer states required capabilities and results, APP declares bindings or adapters, and PHY supplies Java implementations and runtime resources. A complex Java object can internally use hash tables, graphs or numerical algorithms while exposing a clear capability contract.

For example, a delivery quotation may calculate distance from prepared locations and a route network. The business layer explains how distance affects pricing; a technical adapter can reuse Java route-search code. Another search algorithm can replace it without changing quotation rules if input, distance meaning and accuracy remain satisfied. Membership discounts, delivery eligibility and who bears the fee remain in SPP.

This integration needs the appropriate calculation or data-adapter interface. The `before`, `after` and `failure` interceptors in Part V receive invocation metadata only. They cannot replace route calculations or rewrite business participants. This discussion identifies the architectural location of complex implementation bindings in APP; actual binding forms depend on the interfaces provided by the implementation.

Complex requirements can thus be divided between explicit business models and reusable technical implementations. Their complexity does not require algorithms, data structures and deployment details all to enter the business language. Review whether responsibilities are clear, composition complete and each layer fulfills its contract.

Review Model Meaning

Each role explains player, behavior and own facts. Each contract identifies activity, parties and terms. Fields have owners and purposes; business credentials and technical identity have their places.

List-wrapper roles and generic-return contracts need reconsideration. Independently maintained duplicate properties need explicit ownership.

Review Composition

Services need clear duties; scenarios understandable dependencies and completion. Before parallel, check inputs, read/write conflicts and failure joins; before transaction, scope/resources; after phased execution, recovery owners.

Ordinary check failure is not automatic rollback; REQUIRED is not separate commit; co-location does not change domains; remote failure does not prove no transaction occurred. Explain these concrete effects.

Review Benefits and Costs

Roles and declarations reduce mixed logic but concentrate mapping, metadata and plans in tools/runtime. Teams must maintain them and understand output.

Simple business may not justify extra layers. Multiple-party cooperation, complex rules, conservation and environments benefit from explicit contracts. Decide according to real team/system needs.

Review Evidence Levels

Parsing proves accepted expression, compilation checked artifacts, running tests exercised paths, and faults/restarts specific recovery conditions. Performance also needs workload/resources.

Separating levels avoids claiming all reliability from one build. Maintain limits with versions so promises match capability.

Keep Architecture Explainable

Trace each activity through identity, behavior, contract, ownership and result. Locate unfinished phases on failure, responsibility on change and critical paths on optimization.

This book connects participants and behavior through roles/services into scenarios, then APP/PHY into running systems. That connection also guides continuous review and evolution.

Appendices

Appendices A–D develop definitions, assumptions, derivations and counterexamples for deeper reading. E provides language/runtime references; F sources and evidence. All equations use plain text.

Appendix A — The Service Model and Formal Derivations

“Service = participants × behavior” starts business analysis. We explain symbols with a transfer, then decomposition, composition and the precise meanings of equality. Equations use ordinary text readable in Markdown.

A1 Participants and Behavior Jointly Determine an Activity

A transfers 100 US dollars to B. Participant configuration p includes both and the agreement; behavior b defines debit, credit, bounds and results:

```
B = p × b
p = {source: A, destination: B, agreement: 100 US dollars}
b = complete the agreed transfer
B = the business activity of A transferring 100 US dollars to B
```

p is not an amount vector, and b not a number for arithmetic multiplication. The product expresses constitution. Replacing B with C changes the activity; replacing transfer with balance query changes objectives and results.

“If either is missing the result is zero” means no defined activity forms without necessary participants or behavior. Missing payee is not a successful zero-amount transfer. Rejection differs from numerical zero.

A2 Define What Is Compared

State x contains relevant participant facts. For identified A and B:

```
x = (A balance, B balance)
x₀ = (1000, 200)
Exec(B, x) = (resulting state, result classification, observable records)
```

Success gives (900, 300) and a record connected to this transaction. Rejection preserves (1000, 200) with its reason. Communication failure may temporarily prevent a definite answer and require actual-state lookup.

Business equivalence must specify records. Equal funds with different receipts is weaker than equal original associations and replay results. Service contracts choose observations; proofs cannot silently narrow them.

A3 Composition of a Complete Service

Here complete service uses the broad traditional meaning, such as order payment: queries, ownership, amount preparation, transfer and records. Addition denotes activity composition:

```
S = B₁ + B₂ + … + Bₙ = Σ Bᵢ
```

S is whole business and Σ composition. In SPP, services carry individual responsibilities and scenarios organize them. The equation does not require an ordinary service containing all calls. Dependencies, commit boundaries and failures remain part of composition; names alone do not specify execution.

“Transfer + order save” with joint commit differs from fund-first commit and later recovery. Success objectives may match, while failure contracts and guarantees differ.

Minimum Services and Composition Boundaries

Quantum minimum is conservation between two quantum properties; other minimums are role operations. Field reads, expressions and local assignments stay internal rather than becoming smaller business services.

Auxiliary inputs differ from operation targets. A line calculation can use prepared pricing rules but produces a line-role amount. A transfer contract supplies amount while conservation still concerns the balance pair. Collections, calculation reuse and request merging can cover many units without changing the minimum.

A4 Distribution over Behavior

If $b_1 + b_2$ means explicitly organized behaviors on the same participants:

```
p × (b₁ + b₂) = (p × b₁) + (p × b₂)
```

Left names the whole; right expands responsibilities. Equality requires the same bindings, inputs, sequence, boundaries and observations.

For pricing two lines (60, 2) and (20, 3):

```
Line 1 = 60 × 2 = 120
Line 2 = 20 × 3 = 60
Subtotal = 120 + 60 = 180
```

Expansion into two prices and aggregation preserves 180 because aggregation reads exactly their results. Moving aggregation earlier or changing per-line rounding requires rechecking.

Distribution explains expansion, not commutativity. Aggregation waiting for prices is a counterexample to arbitrary exchange.

A5 Distribution over Participants Describes Contributions

A contributes -100 and B +100:

```
δ_A = (-100, 0)
δ_B = (0, +100)
δ = δ_A + δ_B = (-100, +100)
Result = (1000, 200) + (-100, +100) = (900, 300)
(p_A + p_B) × b_transfer = p_A × b_transfer + p_B × b_transfer
```

Each right-hand term is this behavior's effect on its party, not an independent complete transfer. Otherwise the right side transfers twice.

Contributions can be analyzed separately while commit honors the whole. A-only completion (900, 200) violates conservation. Vector addition does not replace a joint-settlement protocol.

Joint eligibility also limits distribution: two jointly qualified applicants need not each qualify alone. Distinguish local contributions from whole-only rules before decomposition.

A6 Quantum Commutativity

For prepared identities and deltas δ_A and δ_B, when both settle:

```
A then B: (x + δ_A) + δ_B
B then A: (x + δ_B) + δ_A
```

Addition makes results equal. Both can affect one property: 1000 minus 100 and 200 gives 700 in either order. The basis is fixed-delta commutativity, not disjoint writes. Appendix B0 gives details.

General queries, checks and calculations still need dependency analysis; Appendix B treats them separately. Bound rejection and recovery require execution contracts.

A7 Testing Business/Technology Separation

Let F be the logical business transformation, G its implementation, and π the mapping from physical records to business facts. Successful semantics require:

```
π(G(physical input)) = F(π(physical input))
```

A cents column maps 10000 to 100 US dollars. A 10-dollar transfer must move 1000 cents, not ten. The mapping then preserves the formula.

Full contracts also preserve identity, bounds, failure and duplicate handling. Not changing the business objective does not make technical duration or responsibility zero.

A8 Constructing a Role-Behavior Model from Object Execution

This section concerns the expressiveness of a modeling method. It does not establish that the current SPP compiler supports every general-purpose program, or that either representation is shorter or faster. Complexity here means that structures, rules and execution steps can continue to compose; it does not mean executing an infinite process completely in finite time.

Consider an explicitly defined object model consisting of object states, operations and their composition. Let state x include object identities, relevant fields and execution position, and let a identify an operation and its

inputs. One execution step is:

```
Object model: x --a--> x′
```

The state must include facts that affect later execution. A hash table includes not only key/value records but capacity and collision rules. If resizing affects later positions, the new capacity is part of the state. If the original model permits alternative next steps, as with concurrent scheduling, this notation denotes a transition relation rather than a single function.

Construct a role behavior for each object operation. Its target becomes a participating role; input values become necessary participants or behavior inputs; state changes follow the original rules. An operation involving several objects lets them play their corresponding roles. Method calls, conditional choices and repetition become behavior composition, branches and repetition.

Let E encode object-model states as role-model states and b_a be the constructed behavior. Each step must satisfy:

```
Original step:      x --a--> x′
Corresponding step: E(x) --b_a--> E(x′)

Business or algorithm observations:
Obs_role(E(x)) = Obs_object(x)
```

Induction then covers every finite execution sequence:

1. E encodes the initial state with equal relevant facts and observations.
2. Suppose the representations correspond after step k. The behavior for operation k+1 was constructed from its original rules, so the resulting state and observations still correspond.
3. After any finite number of steps, the role model therefore continues to express the process already described by the object model.

If an object-based hash insertion encounters a collision at slot 2, checks slot 3 and writes a record, the role model follows the same steps. If the object model resizes and rearranges records first, the role model must include that migration and switch. Encoding only the final key/value set would discard facts that affect subsequent behavior.

The construction assumes the ability to preserve required state, express original operation rules, and provide necessary choice, repetition and composition. These algorithmic facilities need not mean remote services, business transactions or commercial contracts. Imposing SPP's domain constraints on general modeling would change the subject of the argument.

A reverse construction is also possible: objects hold role-model participant state, methods implement behavior, and coordinating objects organize composition. The two modeling starting points can therefore carry the same computable process under these conditions. Clarity and implementation cost remain separate questions.

A9 Expressiveness, Domain Brevity and Implementation Capability

The preceding argument is conditional and constructive. Without explaining how state, rules and composition are expressed, merely observing that no counterexample has been found is not a universal proof. A counterexample must likewise identify a missing expressive capability, rather than show that one notation is inconvenient.

Simulation of the real world first needs an observation scope. A chess model records rules, positions and outcomes; it need not also record the material of each piece or every molecule's motion. Simulation here means preserving specified observations between models. Changing the modeling starting point does not make infinite-precision continuous facts or noncomputable rules fully manageable by finite computation.

Three questions can now be separated:

Question	What Must Be Established	What Does Not Follow Automatically
Can the modeling method express a complex process?	Constructions for state, change and composition, with corresponding observations	The current compiler implements every required capability
Can the domain architecture express commercial services concisely?	How roles, services and scenarios reuse recurring domain structures	Every scientific algorithm fits a few business declarations
Can current tools execute a requirement?	Syntax, binding interfaces, representation, execution and resources	A valid model alone guarantees performance and reliability

S++ obtains much of its brevity from common commercial-service capabilities. Once payment participants, amounts and processing relationships are declared, tools provide standard queries, updates, identity maintenance and technical integration. A new route-search algorithm still has its own complexity; existing Java code can implement it without expanding it into many business declarations.

For external calculation reuse, let C denote the business-required result, J the adapter and implementation, and encode/decode the input/output mappings. The contract requires:

```
decode(J(encode(business input))) = C(business input)
```

The agreement must also cover precision, missing input and failure semantics. A route-search implementation cannot return straight-line distance when road distance is required and still claim equivalence. Missing road data cannot silently produce a zero distance. Approximation needs a contractual error bound; comparison then asks whether the result satisfies that contract, rather than demanding exact numerical equality.

A pure calculation implementation must not hide data queries, persistence or transactional side effects, or the scenario's dependencies and boundaries become incomplete. Required external facts should first be prepared through queries and then passed to calculation. Persistence remains an update organized by the scenario. Reusing general algorithms must not conceal business responsibilities inside technical adapters.

A complex requirement can therefore use roles for business modeling, APP bindings and adapters for existing capabilities, and PHY for implementations and resources. Missing support in a particular binding interface is an implementation gap; it does not prove that the modeling method cannot express the requirement.

Sources: S++ concepts (maintainer reference: `../../../spp/docs/cn/concept.md`), numeric semantics and participant execution. Cross-document issues are tracked outside the manuscript in known issues.

Appendix B — Conditions for Exchange, Reordering and Parallelism

We first prove fixed quantum-delta commutativity, including changes to the same property. We separately prove independent read/write conditions for general activities, then discuss implementation, bounds and failures.

B0 Quantum-Service Commutativity

Let x be a state vector within one business dimension. A and B express prepared δ_A and δ_B. Identities, amounts and units are fixed; intermediate balances do not redefine deltas. If both settle:

```
F_A(x) = x + δ_A
F_B(x) = x + δ_B
A then B: F_B(F_A(x)) = (x + δ_A) + δ_B
B then A: F_A(F_B(x)) = (x + δ_B) + δ_A
By associativity and commutativity of addition:
(x + δ_A) + δ_B = (x + δ_B) + δ_A
```

Both orders produce the same state even with overlapping coordinates. From 1000, -100 and -200 both give 700. Transfers from A to B and C can be written:

```
x = (1000, 0, 0)
δ_A = (-100, +100, 0)
δ_B = (-200, 0, +200)
Either final state = (700, 100, 200)
```

Each delta and their joint result conserve. Intermediate states can differ, but settled totals cannot. Using intermediate values to recalculate amounts changes the delta set being compared.

For n changes and any permutation π:

```
x_final = x + Σ δᵢ = x + δ_π(1) + … + δ_π(n)
```

Every permutation is a finite sequence of adjacent exchanges, each preserving results. This compares the same settled set. Which competing requests settle and how faults avoid lost/duplicate deltas are separate contracts, treated in B7–B9.

B1 Fact Granularity

View state as a map from fact locations to values:

```
(buyer A, tier) → VIP
(goods P, price) → 60
(order O, discount) → 12
```

Different objects of one type are different locations; properties of one object may also be distinct. Include actual mapper completion: copying a whole object writes more than one visibly used field.

R_A contains every location possibly read by A, W_A every changed location. Reads in conditions count. Resolve aliases before deciding whether names denote the same fact.

B2 Assumptions

The following proof assumes:

1. A and B are deterministic, terminating and successful over the input range; rejection and faults are separate.
2. R and W cover every effect, including conditions, mapping and hidden access.
3. Activities use only their read facts and modify only declared writes, with no additional external effects.
4. No third activity changes relevant inputs during comparison.
5. Implementation correctly realizes independent logical writes, with final observation after both finish.

The last excludes corrupting adjacent fields, partial-object reads or undefined memory behavior as substitutes for the logical model.

B3 Sufficient Independence Conditions

```
W_A ∩ R_B = ∅   B does not read A's writes
W_B ∩ R_A = ∅   A does not read B's writes
W_A ∩ W_B = ∅   no shared writes
```

∩ denotes intersection and ∅ the empty set. Read sets may overlap; both reading one stable price does not overwrite it.

B4 Step-by-Step Proof for Successful States

Start with any qualifying x. Let a be the result of A alone and b of B alone.

First, A modifies only W_A, disjoint from R_B. B reads the same facts in a as in x.

Second, B is deterministic, so preceding A does not change B's calculated writes. Symmetrically, preceding B does not change A's writes.

Third, compare each final position:

Location	A then B	B then A
In W_A	A's value, not overwritten	The same A value
In W_B	The same B value	B's value, not overwritten
Outside both	Original x	Original x

Disjoint writes assign each location exactly one category. Final states are equal. Deterministic outputs from those facts are equal too, completing the proof within assumptions.

B5 Why Parallel Execution Can Be Considered

Neither activity changes the other's reads, and disjoint writes do not overwrite. Correctly protected interleavings can therefore produce the same final state.

Business must not observe unfinished whole results. Two fields may be updated at different moments, exposing a partial record. Require publication after joining, appropriate isolation or common commit if only complete results are visible.

The proof does not automatically supply thread safety or atomicity. It establishes logical permission; runtime supplies scheduling, writes and publication.

B6 Counterexample with a Dependency

A sets line amount to 120; B copies current line amount into subtotal. Both initially zero:

```
A then B: line = 120, subtotal = 120
B then A: line = 120, subtotal = 0
```

A's writes intersect B's reads. Concurrent B may likewise read zero. Independent line calculations can overlap, but summation waits.

B7 Commutative Mathematics, Lost Implementation Updates

A adds ten and B twenty to 100. Mathematics gives 130 in either order; read-then-write can lose a delta:

Time	A	B	Balance
1	Read 100		100
2		Read 100	100
3	Write 110		110
4		Write 120	120

A's ten is overwritten. Atomic deltas, exclusion or appropriate isolation prevent this only when implemented. Shared writes do not satisfy B3; other safe concurrency needs another proof.

B8 Bounds Change Accepted Results

From 100 with no overdraft, A debits 80 and B 30:

```
A first: A succeeds, balance 20; B rejects; final 20.
B first: B succeeds, balance 70; A rejects; final 70.
```

The algebraic total is order-independent, but acceptance and final balance are not. A contract may permit settlement order to arbitrate competition, but cannot claim each request's result is independent of order.

B9 Failures and External Observations

A and B write disjoint parties. If A commits then B fails, A remains; if B fails first and stops the scenario, A never occurs. Successful-path exchange does not prove equivalent failure processes.

Joint success needs common commit or recovery. Permitted partial results need explicit states. A parallel fault requires knowing which other branches completed, not merely terminating a waiting thread.

Notifications are observations too. Announcing success after debit but before credit exposes an incomplete promise. Publish after confirmation, with duplication and delivery handling.

B10 Joining Parallel Validations

Two validations reading stable facts without mutation can return independently. If both must pass:

```
Continue = A passes AND B passes
```

AND is order-independent. Choosing the first completed rejection makes displayed reasons schedule-dependent. Fixed priority requires collecting and selecting results accordingly. Immediate-stop control must specify scope and effects on other branches.

This defines the contract needed, not a claim arbitrary bare checks already have it. Appendix E and editorial records state expression limits.

B11 Extending to Groups

Build read/write sets and dependency edges for production, conflicts, observation and commit. Apply sufficient independence to unrelated nodes; legal sequences satisfy all prerequisites.

Two legal sequences differing only in independent-node order can be transformed by adjacent independent exchanges. Each preserves successful results. Shared writes, dynamic aliases or external state prevent direct use of this corollary.

B12 Ordinary Updates with Complete Inputs Commute

Within one transaction's save phase, built-in updates contain no business queries, calculation or checks. Queries/calculations and quantum changes have prepared facts. Any post-settlement calculation/check completes first in serial groups. Let P be complete facts to save; U_i persists its part P_i.

Writes do not depend on another save's result. Different services save different locations; if they overlap, both save the same already fixed P value.

Location k	Final Value in Either Order
Only A saves	P_A(k)
Only B saves	P_B(k)
Both save	P_A(k) = P_B(k) = P(k)
Neither saves	Original value

Thus:

```
U_B(U_A(x; P); P) = U_A(U_B(x; P); P)
```

Adjacent exchange extends to any number, allowing concurrent saves after quantum work followed by a join. Physical implementation still must share transaction context, protect writes and honor common commit or recovery.

Prepared order status, amount and receipt can save independently. Receipt calculations based on settlement must complete first; this proof exchanges ready saves, not calculation with saving.

Built-in updates prevent hidden business access/calls. Serial groups enforce preparation. Complete input and consistent save relations support the proof. Same-identity insert/delete conflicts, foreign-key ordering and physical concurrency are editorial KI-011.

Different transactions can have different P and still require version/conflict protection. Intra-boundary exchange does not prove arbitrary cross-transaction overwrite commutativity.

B13 Why Quantum Work Can Move Earlier

For quantum Q and ordinary U within one boundary:

1. The scenario prepares Q's identity, deltas and bounds; U does not complete or change them.
2. U saves facts, does not produce Q's business calculations and does not modify quantum properties.
3. Q's fixed deltas satisfy B0.
4. Q completes settlement facts; required further calculations/checks run in serial groups before U under B12.

Q therefore has no business need to await U. Delaying saves does not affect Q's inputs. Saves may need Q results, so dependency points from quantum results to saving. The compiler can concentrate Q first.

```
Prepare scenario inputs
          ↓
Q: concentrated quantum execution, unordered or merged
          ↓
Complete settlement facts
          ↓
Serial group: further calculations/checks as needed
          ↓
U: concurrent ordinary saves
          ↓
Join and complete under the transaction contract
```

Commutativity explains Q's internal order, complete-input saving U's internal order, and one-way dependency cross-category scheduling. Quantum-first planning retains explicit settlement-calculation-validation-save dependencies.

Reordering stays within one boundary, without merging independent transactions or turning physical phases into atomic commit. LOCAL can commit common resources; phased backends retain protocols. Editorial KI-012 tracks implementation coverage.

Sources: participant execution, transactions/storage and scenario composition design (maintainer reference: `../../scenario/docs/Composit.docx`).

Appendix C — Performance Models and Experimental Methods

Starting with one call, we derive serial, parallel and merged execution, then critical paths, capacity, completion probability and experiments. All numeric examples are hypothetical, not Petstore measurements.

C1 Cost of One Call

Along its actual path:

$$l_i = c_i + e_i + q_i + t_i + d_i$$

Symbol	Meaning
c_i	Transfer and round-trip cost
e_i	Encoding, decoding and adaptation
q_i	Queueing for service, connections and resources
t_i	Net business work and resource access
d_i	Uncounted transaction setup, coordination and commit

Do not double-count. If server duration includes commit, decompose it or use whole l_i without another d_i. Overlapping intervals require path analysis, not simple addition.

C2 Why Serial Calls Accumulate

If each call begins after the preceding complete return, intervals do not overlap:

$$T_{serial} = h + \Sigma\, l_i + u$$

h prepares common input; u is shared postprocessing/commit. Do not recount branch transactions already in d_i.

With equal c and d, omitting other costs:

```
T_fine_serial = Σ(c + d + tᵢ) = n(c + d) + Σ tᵢ
T_one_call = c + d + Σ tᵢ
Difference = (n − 1)(c + d)
```

The difference comes from repeated boundaries. It assumes comparable work/results and genuinely mergeable communication and commits. Independent business phases cannot be merged just to lower a formula.

C3 Independent Parallel Branches

With simultaneous starts, ready inputs, no mutual waits and no extra contention, completion is the last branch finish:

```
T_parallel = h + max(l₁, l₂, …, lₙ) + j + u
Savings = Σ lᵢ − max(lᵢ) − j
Net gain requires: j < Σ lᵢ − max(lᵢ)
```

j is additional scheduling/join overhead. Two 1-ms tasks with 3-ms overhead take 2 ms serially but 4 ms in parallel. Small work may not justify it.

C4 One Communication and One Commit

If one entry carries independent internal work, resources suffice and actual common commit exists:

```
T_combined_parallel = c + d + max(tᵢ) + j
```

This reduces boundaries and overlaps work. Remote branches still have their own communication, and phased quantum/ordinary commits retain phase costs. SPP syntax alone cannot remove them.

C5 A Complete Numeric Example

Let h = 5 ms, l_1 = 40, l_2 = 70, j = 3 and u = 10:

```
T_serial = 5 + 40 + 70 + 10 = 125 ms
T_parallel = 5 + max(40, 70) + 3 + 10 = 88 ms
Savings = 37 ms
Speedup = 125 / 88 ≈ 1.42
```

If contention increases l_2 to 110:

```
T_parallel_contended = 5 + 110 + 3 + 10 = 128 ms
```

It becomes slower than original serial execution. Measure actual branch times rather than retaining unchanged-duration assumptions.

C6 Dependency Graphs and Critical Paths

For an acyclic graph with duration t(v), unlimited resources and no scheduling overhead:

```
E(v) = t(v)                          if v has no predecessors
E(v) = t(v) + max{E(p)}               for predecessors p
T_ideal = max{E(v)}                  over terminal nodes
```

A node waits for every predecessor; the latest determines its start. Computing E topologically gives the longest dependency path.

Queries of 40 and 70, quotation of ten and settlement of twenty yield:

```
E_buyer = 40
E_goods = 70
E_quote = max(40, 70) + 10 = 80
E_settlement = 80 + 20 = 100
```

Scheduling, queues and retries add or change durations. Improving the 40-ms branch cannot shorten the critical 70-ms branch.

C7 Lower Bounds with Finite Resources

For homogeneous resource work W and k equivalent slots:

```
T ≥ max(CP, W/k)
```

Critical-path activities cannot overlap, requiring CP. Over interval T, k slots perform at most k×T work, which must reach W.

Do not treat all network waits as CPU work or database connections as CPU slots. Model heterogeneous bottlenecks separately; thread count is not automatically k.

C8 Latency and Throughput

Parallelism can reduce one request's time while consuming several connections. If database work capacity is fixed and work unchanged, throughput may not rise.

Under stable load and a fixed boundary, average in-flight L, throughput λ and average residence T satisfy L = λ×T. Summing occupied time over a window and averaging by requests/time explains the relationship. It requires stability, not a continually growing queue, and averages rather than high percentiles.

Measure latency, throughput, concurrency and utilization together. One-request improvement does not establish capacity.

C9 Comparable Experiments

Keep inputs, scale, relationships, eligibility, commit guarantees, protocols and resources equal. Change only the studied factor. After warmup, alternate repeated trials to avoid cache or JIT favoring a fixed order.

Serial/parallel comparison keeps deployment unchanged; local/remote is a separate experiment. Do not simultaneously alter threads, storage, commits and rules and attribute everything to parallelism.

Observe critical paths at low load, then increase concurrency to locate queueing, rejection and saturation. Branch start/end events verify overlap. Artificial delays establish scheduling, not real workload performance.

C10 Metrics and Reports

Keep request start/end, result, retries and branch associations. Report median and percentiles from sorted durations; nearest-rank p95 is item ceil(0.95×N), with the method stated.

Report success, business rejection, technical failure and timeout separately. Do not discard slow failures and report success averages alone. Define throughput as completed requests or successful business, and distinguish replay from new settlement.

Retain versions, build, resources, workloads, warmup, rounds, correctness, raw data and analysis. Functional regression is not this benchmark.

C11 Remote Dependencies and Whole-Business Completion

This section asks whether valid business completes within a deadline. Eligibility rejection is not technical unavailability, and per-request completion probability is not time-based availability.

If every remote call must succeed, with S_i its in-deadline success:

```
P(completion) = P(S₁ ∩ S₂ ∩ … ∩ Sₙ)
```

Only with independence, no retry/alternative/partial-completion policy and equal p:

```
P(completion) = pⁿ
p = 0.999, n = 20:
```

```
pⁿ ≈ 0.980189 ≈ 98.02%
```

For 0 < p < 1, each required call multiplies probability by a factor below one. This proves cost in that model, not universal incompatibility of microservice scalability and availability.

Shared networks, databases and hosts invalidate independence. Use joint probability or measurement instead. Retry, redundancy, isolation, alternatives and recovery change event definitions and structure and require new models.

C12 What Concurrency, Merging and Local Deployment Change

Serial and parallel require the same intersection if scheduling does not change success conditions. Starting order alone does not remove branch faults. Parallelism mainly reduces time; fixed deadlines may see fewer accumulated-time timeouts, not automatic availability improvement.

Merged requests reduce communication and protocol boundaries. Multiple deltas to one executor can arrive together, reducing request-level failures and confirmation cost. The executor still must apply every change; one entry does not make internal resources infallible.

Local calculation/checks remove separate remote dependencies without changing logical service count. Host and storage still fail, requiring their own statistics. Do not replace p^n by p and call it measured improvement.

The book's latency mechanisms are exposed independence, quantum request merging and local small operations. They do not prove SPP fixes remote-dependency availability.

Use C9–C10 with equal guarantees/resources, measuring remote calls, timeouts and failures. Keep slow failures in statistics.

C13 Property-Use Analysis and Multilevel Caching

Determining Query Sets

For role R, Uses(R) includes properties read by scenarios, calculations, validations, quantum calls and bound expressions within the compilation scope. Follow calculation calls, mappings and relationships, not merely direct expressions.

Queries also locate and confirm identity and obtain saving baselines:

```
Fields(R) = Uses(R) ∪ NeededIdentity(R) ∪ NeededBaseline(R)
```

Outputs count as use too. APP and output-protocol dependencies need analyzable implementation or explicit declarations. Without evidence, choose conservative fields rather than treating unrecognized access as absent.

Mapped attributes require source dependency sets: Buyer tier may come from a role and address from a root entity. Preserve identity/mapping/association requirements; on-demand querying is not arbitrarily skipping necessary role relationships.

Correctness requires every property to be completed from its source before use. Unloaded differs from actual null or zero. Pruning must not make defaults business facts or replace stable input participants.

Partial Queries and Saving

Saving must not overwrite unloaded facts with defaults. Loaded-range metadata, explicit writes or another preservation mechanism can protect them; valid baselines still support versions.

Analysis, reading, mapping and saving jointly implement optimization. Fewer SELECT columns or response fields alone do not prove whole-business equivalence.

Cache-Hit Conditions

Request caches reduce repeated scenario reads; process caches reuse stable facts across requests. Remote source facts can be locally cached then mapped.

Keys include source domain, identity and isolation conditions, not only type or incomplete numbers. Records state loaded sets. If requested fields F and cached C satisfy $F \subseteq C$, field coverage qualifies; otherwise fetch missing fields or refresh.

Versions, notifications or expiration offer different freshness guarantees chosen by requirements. Missing identity, technical read failure and expiry differ; old confirmation cannot be reused unconditionally.

Cache speeds retrieval, not actual quantum bounds/conservation. Display balances can be cached, while settlement uses authoritative facts. Protected ordinary transaction reads also need appropriate version, isolation and baseline conditions.

Gains and Verification

Pruning reduces transfer, reading and mapping but does not guarantee scan time proportional to columns. Cache savings depend on hits, refresh, size and contention.

Keep roles, rules and save results fixed. Record fields, payload, source calls, hits, latency and refresh costs. Test missing dependencies, partial hits, source changes, conflicts and saving unloaded fields to ensure unchanged meaning.

These are methods and correctness conditions, not acceptance evidence that current compiler implements generic pruning/caching. Editorial KI-014 tracks scope.

Sources: S++ concepts (maintainer reference: `../../../spp/docs/cn/concept.md`) and deployment configuration.

Appendix D — Quantum Services and Conservation Mechanisms

Balances and inventory explain deltas, conservation, bounds, atomic settlement and inverses. Each conclusion separates mathematical grounds from execution conditions, avoiding conservation arithmetic as a substitute for commit guarantees.

D1 Changes as Vectors

The minimum quantum service is a pair of properties with equal opposite changes in one dimension. A larger vector can embed that pair or combine several services. State dimension differs from minimum service granularity.

For accounts A, B and C, A transfers 100 to B:

```
x = (1000, 200, 50)
δ = (−100, +100, 0)
x' = x + δ = (900, 300, 50)
```

Coordinates identify complete participants and properties, not merely “balance.” Units and precision must agree; inventory coordinates are per goods identity.

D2 Conservation Proof

Define total in one dimension:

```
Q(x) = Σ xᵢ
Q(x + δ) = Σ(xᵢ + δᵢ) = Σ xᵢ + Σ δᵢ
Σ δᵢ = 0  ⇒  Q(x + δ) = Q(x)
```

Before and after totals are equal exactly when deltas sum to zero. −100 +100 +0 qualifies; debit 100 and credit 99 leave −1. A fee transferred to another account requires that real destination in the model.

This proves the applied complete vector. Crashing after debit before credit can still violate actual conservation, so joint commit is necessary.

D3 Prove Independent Dimensions Separately

For available, reserved, sold and supply of goods P:

```
After receipt:       (10, 0, 0, -10)
After reserving 2:    (8, 2, 0, -10)
After shipping 2:     (8, 0, 2, -10)
```

Each sums to zero. Goods Q needs a separate equation; one missing P and extra Q do not prove P correct. Money and item counts cannot be added. Exchange and unit conversion require rates, units, differences and rounding policies.

D4 Prepared Inputs and Actual Bounds

The caller prepares fixed amount a and postsettlement bounds. Execution uses current x, not an earlier snapshot:

```
Candidate xᵢ' = xᵢ + δᵢ
Require Lᵢ ≤ xᵢ' ≤ Uᵢ
```

null disables the corresponding side. Participant/contract facts determine limits before invocation; execution handles dynamic balances.

Two debits of 80 may both see 100. After one leaves 20, the other's -60 candidate rejects. Checking and protected writing belong to one mechanism, not a check followed by unprotected write.

D5 Whole-Group Settlement

Success applies the entire δ; rejection applies none of the group. Partial unfinished state must not be published as success.

A local backend can protect reads, calculate, validate and write in one database transaction. Actual connection, isolation or atomic conditions and transaction capability determine guarantees.

Cross-resource execution needs coordination/recovery. A declared boundary does not coordinate independent connections automatically. Later ordinary failure cannot locally roll back already committed quantum phases.

D6 Merged Submission and Delta Aggregation

Prepared services have order-independent joint results under B0, so one request can carry several without sequential send-and-wait order. The executor arranges changes and common submission.

A merged request can preserve individual changes, identities, bounds and result associations. If allowed, aggregate by complete identity and dimension:

```
A → B 100: δ₁ = (−100, +100, 0)
B → C 30:  δ₂ = (0, −30, +30)
```

```
Net δ = δ₁ + δ₂ = (−100, +70, +30)
Net sum = 0
```

Zero-sum deltas retain conservation after aggregation. Communication merging differs from netting: one request can retain original details. Per-step and net-value bounds need not agree.

From (0,0), A→B100 then B→A100 nets zero. Final-only checks pass; non-overdraft checks reject the first step. The contract must select the semantic boundary instead of assuming conservation permits temporary overdraft.

With n changes, fixed request overhead h and per-change transmission p:

```
Separate communication = n × h + n × p
Merged communication = h + n × p
Fixed overhead saved = (n − 1) × h
```

Data still travels. This is not system timing prediction: size, response details, sharding and protocol affect savings. One submitted request may still cause backend cross-resource communication.

Keep duplicate detection, original interpretation and recovery evidence. Net zero does not mean no business occurred. The objective is reduced communication without altered meaning or guarantees.

D7 Request Identity and Idempotency

Identity binds immutable amounts, endpoints, group and bounds. Successful same-identity retries return original results without reapplying δ; changed facts conflict.

Durable identity/results must relate reliably to actual writes. Crashing between balance and dedupe recording can duplicate credit; recording success first can claim a nonexistent update. Backends must resolve this commit relationship.

D8 Why Inverse Transactions Preserve Later Changes

Original x, settled δ and later net e give:

```
Before inverse = x + δ + e
After inverse = x + δ + e − δ = x + e
```

Thus e remains. 100 +50 +20 −50 =120; restoring 100 erases twenty.

The system automatically inverses original records without developer reverse services. It must preserve exact components and identity, additive meaning and deduplication. Assignment or irreversible external effects do not satisfy this proof.

D9 Inverses Versus Subsequent Business

Failure recovery restores original quantum effects. Original location, inverse constraints, durable results, deduplication and recovery are system duties, with APP technical policy.

Later partial refund, reversal or fees are new business. A partial amount or deducted fee differs from -δ and needs its own eligibility/formula. Original inverse -δ must not be confused with a new δ′.

D10 Unknown Outcomes and Retry

Disconnection after commit but before response leaves timeout with possibly changed balance. Stable-identity lookup distinguishes confirmed success, rejection, protocol-confirmed nonexecution and unknown.

Unknown does not mean unexecuted. New identity may transfer twice; immediate compensation may reverse something that never occurred. Confirm under protocol or wait for recovery evidence.

D11 Boundaries and Implementation Coverage

Relation	Required Actual Guarantee
LOCAL	Same JVM, actual database/schema, shared connection/commit resources
Non-LOCAL mixed phases	Defined quantum/ordinary order, durable outcomes and recovery
REQUIRED	Reuse outer boundary, no independent commit
REQUIRES_NEW	Independent resources/propagation and outer-failure handling
Unsupported propagation/protocol	Explicit build/deployment rejection, no silent weakening

Appendices E/F describe example support. Names and passing test counts alone do not prove arbitrary resource combinations.

D12 Value Uncertainty After Settlement and Observation

At preparation the query sees b; this request settles δ. Let C contain other changes effective before later observation. In the same additive dimension:

$$b_{observed} = b + \delta + \Sigma\, e_j \quad (j \in C)$$

Confirming δ without knowing C cannot determine the observed balance from b+δ alone. With b=1000, δ=-100 and another e=-200:

```
Other request not effective: C = ∅, observed 900
Other request effective: C = {e}, observed 700
```

Both assume this debit settled. Superposition here is epistemic uncertainty after settlement and before observation, not two stored balances or physical probabilities/amplitudes. Confirmation does not identify every concurrent change included in observation.

For a fixed settled set:

```
(1000 - 100) - 200 = (1000 - 200) - 100 = 700
```

This does not prove every request can settle. From 100, -80 and -30 cannot both satisfy zero lower bound. Order/arbitration can determine rejection. Candidate states pass through bounds; not every algebraic value is legal.

Confirmation answers whether this delta succeeded; observation what the property is then. A preparation query does not freeze later changes and settlement does not fix b+δ. Observations have explicit read semantics and can be followed by new changes.

Lock-read/calculate/overwrite determines a result during protection. Quantum deltas defer dynamic values and bounds to settlement. This distinction does not prove a lock-free backend: short locks, atomic conditions or coordination may still protect delta application and commit. Evidence establishes actual capability.

Sources: quantum bounds, transaction/storage, environments. Unresolved contracts are outside the manuscript in known issues.

Appendix E — Language and Engineering Reference

This appendix gives language conventions and experiment commands used by the draft. It aids reading; module source and runtime instructions remain authoritative. Run commands at repository root.

Concept Reference

Concept	Responsibility
Entity	Natural entity owned by this domain
Reference	External entity, role or contract with explicit source
Role	Participating identity and role facts

Concept	Responsibility
Contract	Parties, terms and scenario results
take … as	Single, joint or further role playing
mapsame	Explicit compatible same-name mapping; not default
takeall	Import player properties and generate mappings
business	Explain true business use of a suspicious technical ID and remove habitual-ID warning
query	Complete participants and support identity confirmation
check	Judge conditions without mutation
calculate	Prepare candidates and compose pure calculations
update	Apply changes through application/resources
quantum update	Define deltas and settlement relationships
Scenario	Organize complete activities
transaction … fail	Business handling after execution and technical recovery
ResultInfo	Built-in scenario result/recovery participant, not an atomic parameter

Entries use participants or collections; names can default. Argument-free calls require unique type binding. There are no business return values. Scalar input identity stays stable, while mutable collection contents can change.

Ordinary check ends lexical scope and does not default to rollback. Explicit reversal uses the repository spelling `breakTransction`. Participant execution defines conditions and provisional control flow; the text does not assume universal support across transaction modes.

Example Backend Coverage

These are concrete baseline capabilities, not new conceptual definitions:

Feature	Conditions
LOCAL	Same JVM and verified common database/schema/connections
REQUIRED	Reuse execution and commit context
REQUIRES_NEW	Supported homogeneous outer/local combinations; independent results handled separately

Feature	Conditions
NESTED	Rejected by example backend
Bare check in parallel	Limited stop scope; use supported serial or guard forms
Enhanced for	Collection-order iteration, not automatic parallel calculation
calculate calls	Pure calculation composition; no cycles or technical effects
fail	Outer transactions in serial scenarios; fail(targetContract) for multiple contracts; nested/parallel fail rejected
Remote transaction/quantum protocol	Actual coordination and recovery evidence required; otherwise rejected

Execution, storage and environment specifications define exact checks. Principles establish when combinations make sense; backends still need actual runtime support.

Implementation References for Service Categories

Characteristic	Source or Example
Independent concurrent queries followed by fact use	Ordering, payment
In-place completion and identity	Query binding, execution
Built-in query/update; visible scenario rules	Implicit CRUD
Atomic call restrictions and calculation cycles	SppDomain (maintainer reference: `../../lang/src/main/java/free/servpp/lang/models/SppDomain.java`): validateBusinessCalculations, validateCalculationCycles
Calculation-only calls, no check mutation/require	BusinessCalculationValidator (maintainer reference: `../../lang/src/main/java/free/servpp/lang/models/app/expr/BusinessCalculationValidator.java`)
Quotation reuses line calculation	Pricing: quoteOrder and priceLine
Lexical checks, guards and reversal	Execution, atomics

Characteristic	Source or Example
Record rejection after recovery	Reservation, system results
Local calculation/validation generation	SppService (maintainer reference: `../../lang/src/main/java/free/servpp/lang/models/SppService.java`): isLocalService; generator (maintainer reference: `../../generator/src/main/java/free/servpp/generator/openapi/MustacheClassWriter.java`); APP

Concurrency and local sequencing follow duties/dependencies, not automatically service categories. Local execution still accepts concurrent requests. Part I explains common arrangements, Part II costs and dependencies.

Build and Test

```
mvn -pl :petstore-demo -am package -DskipTests -Dspp.environment=develop
mvn -pl :petstore-demo -am test-compile -DskipTests -Dspp.environment=testing
mvn -pl :petstore-demo -am test -Dtest=PetstoreScenarioTest -
Dsurefire.failIfNoSpecifiedTests=false
```

The first two build/compile but skip running tests. The third runs selected tests. package success alone does not prove regression. Latest Petstore acceptance records provide joint-regression commands.

Local Web Application

```
SPP_PETSTORE_JDBC_URL='jdbc:h2:file:/tmp/spp-book-develop' \
  ./demos/petstore/run-demo.sh web
```

Default port is 8086. Existing matching builds can use `SPP_DEMO_SKIP_BUILD=1`. Fixed paths retain previous data; choose paths by experimental round.

Three-Process testing

Build testing artifacts first. Set identical caller credentials and experiment resources in all three terminals, replacing the token placeholder:

```
export SPP_ENVIRONMENT=testing
export SPP_DEMO_SKIP_BUILD=1
export SPP_DOMAIN_RPC_TOKEN='replace-with-session-token'
export SPP_TESTING_CUSTOMER_JDBC_URL='jdbc:h2:file:/tmp/spp-book-testing-customer'
```

```
export SPP_TESTING_PRODUCT_JDBC_URL='jdbc:h2:file:/tmp/spp-book-testing-product'
export SPP_TESTING_SHOP_JDBC_URL='jdbc:h2:file:/tmp/spp-book-testing-shop'
export SPP_TESTING_LEDGER_JDBC_URL='jdbc:h2:file:/tmp/spp-book-testing-ledger'
```

Start providers, confirm readiness, then start shop:

```
# Terminal 1, default port 8087
./demos/petstore/run-demo.sh web customerService
```

```
# Terminal 2, default port 8088
./demos/petstore/run-demo.sh web productService
```

```
# Terminal 3, default port 8086
./demos/petstore/run-demo.sh web petstore
```

Cross-host deployment needs endpoints and quantum resources ensuring one actual system. Identical copied H2 file-path strings do not create a shared remote database.

Numbers and Output

SPP numeric specifications give defaults; PHY replacement and field overrides select representation, with field overrides taking precedence. Capacity is not rounding. Generated output stays in target; source in `src/main/spp`.

See runtime instructions, launcher, execution, numbers and environments.

Appendix F — Sources and Verification Evidence

This appendix provides concepts, source, execution and performance references. Concepts form one system; remaining cross-document issues are separately tracked in editorial records, outside the narrative.

F1 Theory and Methods

Reference	Coverage
S++ Concepts and Applications (maintainer reference: `../../../spp/docs/cn/concept.md`)	Service model, decomposition, quantum services, polymorphism, separation and performance

Reference	Coverage
S++ Modeling (maintainer reference: `../../../spp/docs/cn/modeling.md`)	Shop participants, identity, agreements and storage
Scenario Composition Design (maintainer reference: `../../scenario/docs/Composit.docx`)	Categories, dependency groups and scheduling
Application Framework Design (maintainer reference: `../../lang/doc/ApplicationFrame.docx`)	Runtime, technical handling and transactions

The neighboring spp repository must be obtained separately. Relative source references require that checkout beside this repository.

F2 Language and Engineering Specifications

Topic	Reference
Meaning and participation	Business model boundaries
Query, identity and control	Participant execution
Calculation rules	Atomic calculation
Representation and rounding	Numeric semantics
Quantum bounds	Call bounds
Ownership, commit and recovery	Transaction/storage
Environments and resources	Deployment/physical configuration

Petstore counts retain the separate baseline in F7. Bank source and newly added capabilities are checked against main revision `0fafa21d`. Specifications state goals and constraints, source expresses implementation, and runtime records evidence specific paths. No single layer replaces the others.

F3 Petstore Functional Verification

October 13, 2026 technical-service verification records 1,123 passing tests: language 362, runtime 617, generator 105, Petstore 39, with zero failures, errors or skips. Coverage includes matching, compilation, phases, access rejection, observer isolation, concurrency and replay. Remote proxies use controlled transport, without real cross-JVM acceptance. The Petstore example shows all-service tracing. Capability comes from main revision `65c766fc`; this book cites existing records.

October 13 failure handling records 1,104 develop tests: language 356, runtime 607, generator 105, Petstore 36, all passing without skips. Reservation evidence covers insufficient stock, ordinary-save fault and replay. Framework tests separately establish real JDBC settled inverse before fail. Petstore used LOCAL, not real distributed TCC/external quantum acceptance. These are cited records, not tests rerun for the manuscript.

October 12 final regression records 1,080 passing tests: lang 340, appframe 581, generator 105, appframe-seata 20, Petstore 34; no failures, errors or skips.

That record also covers complete Chrome flows in develop, three-JVM testing with four isolated file databases, and shop restart. After shutdown, checks compare fund totals, per-goods stock/supply and per-transaction delta sums.

Temporary log paths identify original evidence; independent environments must rerun commands to obtain their own results. Known limitations cover opening retries and split zero recovery, detailed in recovery design. Functional regression is not Appendix C's serial/parallel benchmark.

F4 Verification of the Bank Deposit Case

The bank deposit case has completed its migration to current syntax and runtime protocols. Its production source uses only SPP, APP and PHY, with no handwritten production Java. SPP does not use `try/catch/throw`; generated Java and runtime internals can still use exception handling for technical failures. Historical source remains for rule comparison, not as current executable examples.

This section checks the case against main revision `0fafa21d` and cites existing repository evidence. The manuscript update did not rerun bank tests. The latest verification record preserves commands, results and scope by batch:

Verification Batch	Result	Scope
October 13, 2026 clean develop joint regression	1,226 tests; zero failures, errors or skips	Language 366, runtime 628, generator 107, bank case 86, Petstore 39; internal recipient accounts, historical snapshots and included fault/concurrency tests
Subsequent testing bank regression	86 tests; zero failures or errors, two skips	Separate quantum and ordinary H2 data sources in one JVM; two LOCAL-only commit-fault tests skipped
Independent reversal-preview tests	Two passing tests in each of develop and testing	No preview writes; formal reversal preserves later deposits; missing originals are rejected without side effects
Chinese workbench verification	Backend tests and browser flows pass in both environments	Exact decimal display, negative-amount rejection, preview, reversal, receipt lookup and mobile layout; see workbench acceptance

Split-storage testing does not constitute complete acceptance of real network distribution, XA, Seata AT or TCC. Independent JVM restart and forced exit after confirmed settlement cover their specified persistence paths, not

automatic resolution of interrupted or unknown commits. Later targeted tests must not be added together and represented as one unexecuted joint regression.

The completion audit identifies implemented scenarios, storage, transactions and technical capabilities. The 86-entry historical migration inventory lists evidence, differences and unmigrated parts entry by entry. Those 86 historical entries and the 86 JUnit tests above are different measures; equal numbers do not establish one-to-one equivalence.

Known issues retain boundaries for committed ordinary Saga recovery, persisted pending authorization and cross-process resumption, automatic confirmation of unknown settlement, and durable auditing of every invocation. A split-storage failure can restore quantum funds while leaving ordinary committed facts. The system explicitly reports `RECOVERY_REQUIRED` and `handled=false`, not complete rollback.

Source mapping gives current directories and original rule locations. Source derivation, mathematical expectations, demonstration inputs, generated execution comparisons, original COBOL execution and bank-confirmed results represent different evidence levels. Case success does not establish every bank policy or original-system output equivalence.

F5 QService References

The neighboring qservice repository's SPP boundary contract (maintainer reference: `../../../qservice/docs/spp-qservice-boundary-contract.md`) and quantum design (maintainer reference: `../../../qservice/docs/qservice-early-design.md`) describe commands, durable identities, lookup and recovery.

It has separate artifacts and baselines. Verify the backend protocol actually integrated; independent-project tests do not extend to arbitrary deployments.

F6 Using Evidence

Parsing establishes accepted expression; compilation checks artifacts; runtime exercises paths; fault/restart tests cover specified recovery. Performance needs workload/resources.

For each claim, identify participants, behavior and configuration, then input, expectation, result and fault scope. These connections make conceptual arguments reviewable as engineering practice.

F7 Petstore Code-Size Statistics

The figures in this section describe their recorded source baseline. Subsequent English/USD presentation and expanded source formatting change physical line counts; they do not retroactively change these measurements.

On October 13, 2026, source at book branch `5b9b182c` was measured; that manuscript change did not alter demo source. develop generation began with no existing generated Java directory, excluding stale files. Run at root:

```
mvn -pl :petstore-demo -am package -DskipTests -Dspp.environment=develop
```

Inputs are `.spp`, `.enum`, `.app` and `.phy` under `demos/petstore/src/main/spp/`; output is `.java` under `demos/petstore/target/generated-sources/openapi/src/main/java/`. Excluded are tests, pages, resources, build scripts, generator and shared runtime dependencies. Java includes both S++ and OpenAPI generation. Reproduce in a clean directory and recount after source/tool changes.

Category	Files	Physical Lines Including Blanks/Comments	Code Lines
SPP	63	1,104	999
Enums	2	25	18
APP	4	126	117
PHY	4	222	214
Inputs total	73	1,477	1,348
Generated Java	962	131,107	100,151

Models account for 64,431 code lines, services 21,031, interfaces/handlers 14,535, and mapping/configuration 154. Model and request/response expansion is substantial; every line is not independently designed business logic.

This script removes blank, line-comment and block-comment lines while preserving comment symbols in strings and brace-only lines. It fits this source, not every language construct:

```
from pathlib import Path
import re

pattern = re.compile(
    r'"(?:\\.|[^"\\])*"|\'(?:\\.|[^\'\\])*\'|//[^\n]*|/\*[\s\S]*?\*/'
)

def code_lines(source):
    def remove_comment(match):
        token = match.group()
        if token.startswith(("//", "/*")):
            return "\n" * token.count("\n")
        return token
    source = pattern.sub(remove_comment, source)
    return sum(bool(line.strip()) for line in source.splitlines())
```

```
source_root = Path("demos/petstore/src/main/spp")
java_root = Path("demos/petstore/target/generated-sources/openapi/src/main/java")
for suffix in (".spp", ".enum", ".app", ".phy", ".java"):
    root = java_root if suffix == ".java" else source_root
    files = sorted(root.rglob("*" + suffix))
    sources = [path.read_text(encoding="utf-8") for path in files]
    print(suffix, len(files),
          sum(len(source.splitlines()) for source in sources),
          sum(code_lines(source) for source in sources))
```

SPP can place multiple declarations/calls on one line, while Java formatting, accessors and request/response types expand lines. This is version-specific scale, not a universal expressiveness, productivity or speed ratio. Exact handwritten Java comparison requires an independent implementation with equivalent business, capability and acceptance. The following is an estimate by responsibility.

Estimate with Common Java Frameworks

Assume Spring Boot + MyBatis + Lombok, existing reusable components, no separate remote interface for every internal operation and no manually expanded accessors. Lombok provides model boilerplate, MyBatis SQL mapping and Spring declarative transactions. Their source is dependency, not project-maintained code. See Lombok Data, MyBatis Mapper and Spring transactions. Documentation establishes capabilities; estimates are the book's engineering judgment, not vendor benchmarks.

Maintained Content	Estimated Lines	Basis
Models, DTOs and enums	1,000–2,000	Four domains, reused models and Lombok
Entries, conversion and mapping	1,000–2,000	34 scenario boundaries
Orchestration, calculation and eligibility	3,000–5,000	Approximately 90–150 per scenario including shared rules
Persistence, SQL, mappings and schema	2,000–4,000	Facts, snapshots, version conditions and associations
Conservation, idempotency and recovery integration	2,000–5,000	Bilateral deltas, bounds, original confirmation and phase recovery
Domain calls, environments and technical errors	1,000–2,000	Local/remote adaptation, resources and environment selection
Total	10,000–20,000	Java, SQL/Mapper and necessary configuration

Responsibilities are allocated to avoid duplication: business rejection belongs to scenario/checks; timeout confirmation to recovery/technical handling. Proposed general cache pruning and full security/governance not yet implemented in the case are excluded, as is developing the shared framework itself.

A conventional monolith with local transactions, merged models, less domain adaptation and only local rollback may need 6,000–10,000 lines. That narrower scope omits domain deployment and its confirmation/recovery and is not an equivalent-capability baseline.

Mature accounting, idempotency and coordination middleware reduces handwritten code; building such general facilities can exceed these ranges. Formatting also matters. Ten to twenty thousand describes maintenance scale, not an exact productivity multiplier or development time.

F9 COBOL and S++ Code Size in the Bank Case

The count was made on October 14, 2026, using case source and reference documents at repository revision `92ed491b`. Original COBOL and COPYBOOK files were read from the local bank-source directory, without modification or copying their contents into this book. That directory must be obtained separately. File inventories and figures are preserved in the appendix's count results.

Source Scope and Deduplication

The “directly located for analysis” scope selects nine COBOL programs identified by program filename or specific line references in the source documentation. Located analysis means there is a source-tracing basis; it does not mean every branch of each program has been migrated.

Program	Source Responsibility	Physical Lines	Lines Excluding Blanks and Comments
GSA01102	Front-office acceptance of a general deposit	2,325	1,820
KSADEP0	Deposit core, repayment allocation and reversal	4,197	3,508
CSAQRY1	Account eligibility and acceptance checks	2,654	1,867
KSAFEE0	Fee entry point	384	310
CSAFEE0	Fee rules	415	331
CCMDAT1	Calendar and day counting	2,856	2,563
CSAPRD2	Deposit and agreement accrual products	483	315
CSAPRD4	Overdraft accrual products	237	154
GCCBAAR	Accounting-rule integration and configuration-source analysis	605	542
Total	Nine programs	14,156	11,410

Active `COPY` statements in those programs directly reference 135 COPYBOOKs. Nested references add another 26, yielding 161 distinct files. A shared COPYBOOK is counted once by actual file, even when several programs or repeated statements reference it. Invocation and expansion counts do not multiply it.

The broader “all related” scope selects 22 programs explicitly mentioned in the source documents and actually present in the original directory. It includes dispatch, query and configuration programs. It does not recursively expand every `CALL` dependency and is not the size of the entire banking system.

Source Scope	COBOL Programs	Distinct COPYBOOKs	Program Physical Lines	COPYBOOK Physical Lines	Total Physical Lines	Total Lines Excluding Blanks and Comments
Directly located for analysis	9	161	14,156	23,488	37,644	22,589
All related programs mentioned in documentation	22	192	20,626	24,961	45,587	28,358

See the direct-source inventory and all-related-source inventory for per-file figures. They contain locations and counts, not bank source contents.

Current S++ Source

Only production SPP, enums, APP and PHY are counted. Historical material, Java tests, test inputs, web pages, build scripts, generated Java, and shared compilers and runtime frameworks are excluded.

Source Category	Files	Physical Lines, Including Blanks and Comments	Lines Excluding Blanks and Comments
SPP	34	1,037	973
Business enums	1	6	6
APP	2	48	48
PHY	2	38	38
Total	39	1,129	1,065

SPP and enums total 979 nonblank, noncomment lines describing the business models, services, scenarios and finite value sets maintained in this case; APP and PHY add 86. Several S++ statements currently share a physical

line. Line ratios therefore do not directly measure algorithmic complexity, work or productivity.

Counting and Reproduction

Physical lines follow each file's original line breaks. COBOL nonblank, noncomment counts use fixed-format processing: exclude lines whose seventh column is `*` or `/`, take content from columns 8–72, remove inline comments following `*>`, and count remaining nonempty lines. Sequence columns and right-hand identification areas are not code. Data definitions, `COPY` directives, procedures and control declarations still count; these are not business-statement counts.

COPYBOOKs follow the same rule. SPP, enums, APP and PHY remove `//` and `/* … */` comments while preserving comment symbols in strings, then count remaining nonempty physical lines. COPYBOOKs are not expanded and repeatedly counted inside every program.

The reproduction script takes the original COBOL root directory. Run from the repository root:

```
python3 book/附录/统计资料/count-bank-code.py /path/to/original/cobol-root \
  --output-dir /tmp/bank-demo-code-count
```

The root must contain the original program and COPYBOOK category directories. The script writes two source inventories and a summary JSON. Changes to source or reference documents can change scope and results; recount and identify the revision. Neither current scope contains unresolved or ambiguous COPYBOOK references.

Limits of the Comparison

Source and implementation sizes help explain how concentrated the expression is; they do not prove equivalent conversion of every source line. Entire source programs include other transaction modes, shared configuration and unmigrated branches. The S++ case covers business paths explicitly implemented and verified.

Role and service models concentrate their rules, participating relationships and execution boundaries. Tools supply repetitive implementation, and APP/PHY connect storage and runtime facilities. The case still depends on shared frameworks, uses demonstration configuration for accounts and external facts, and retains authorization-resumption and ordinary Saga recovery limits described in F4. Dividing all source lines by case lines does not establish an unconditional compression factor or full replacement.